



MASTER'S DEGREE PROGRAMME

Innovation and Product Management

**Conceptual integration of product variant, product line
and product platform modelling capabilities into
CADdrive**

SUBMITTED AS A MASTER THESIS

to obtain the academic degree of

Master of Science in Engineering (MSc)

by

Aleksandra Erohina

Wels, December 2025

Thesis supervisor

Dr. rer. nat. Georg Hackenberg M.Sc.

ACKNOWLEDGEMENTS

I would like to take this opportunity to express my sincere gratitude to all those who have supported and guided me throughout my academic journey and the completion of this thesis.

First and foremost, I extend my deepest appreciation to my thesis supervisor, Dr. rer. nat. Georg Hackenberg, for his invaluable guidance, engaging discussions, and steady mentorship throughout this research process. His expertise and patience were instrumental in shaping this work.

I would also like to extend my sincere thanks to the faculty of the Innovation and Product Management program. The past two years of study have been a truly enjoyable and enriching journey, and I am grateful to all the professors for their inspiring instruction and dedication.

My profound gratitude goes to my family, whose unwavering support was my foundation. To my husband, Andrey, thank you for your non-stop encouragement, understanding, and for being my pillar of strength. To my daughter, Polina, thank you for your boundless motivation and for reminding me of what truly matters.

I am also deeply thankful to my sister, Anzhelika, for her invaluable guidance and steadfast belief in me. A special thanks to my brother-in-law, Dávid, for his incredible readiness to help at a moment's notice.

This achievement is shared with all of you.

SWORN DECLARATION

I hereby declare that I prepared this work independently and without help from third parties, that I did not use sources other than the ones referenced and that I have indicated passages taken from those sources.

This thesis was not previously submitted in identical or similar form to any other examination board, nor was it published.

This printed thesis is identical with the electronic version submitted.

.....

Aleksandra Erohina

Wels, December 2025

ABSTRACT

The shift towards mass customization demands efficient methods for developing product families rather than single products. Software Product Line Engineering (SPLE) provides a proven framework for this, enabling systematic reuse and management of variability across a family of products. However, existing SPLE tools are often complex and costly, making them inaccessible for education. This thesis addresses this gap by conceptualizing the integration of product line engineering capabilities into the open-source, educational platform CADdrive.

The research explores how core SPLE principles, such as variability modeling and the use of 150% models, can be adapted for configuring physical products using digital LEGO as a low-fidelity tool. A structured methodology includes a literature review, a case study of a drone product line, and a prototype of a user interface within CADdrive. The central research questions investigate the adaptation of SPLE techniques for digital LEGO, leading to the creation of a feature model, a 150% LEGO model, and an algorithm for the automatic derivation of product variants. A key innovation is the method of combining the graphical feature model with the 3D model by expressing it as a formula embedded within the LDraw file structure.

The proposed conceptual framework demonstrates that CADdrive can be successfully extended into a practical educational tool for product line engineers. It provides users with a hands-on environment to learn how to design, manage, and derive product variants, thereby bridging the gap between theoretical SPLE concepts and their application in physical product development.

KURZFASSUNG

Der Wandel hin zur Massenanpassung erfordert effiziente Methoden für die Entwicklung von Produktfamilien anstelle von Einzelprodukten. Software Product Line Engineering (SPLE) bietet hierfür einen bewährten Rahmen, der die systematische Wiederverwendung und Verwaltung von Variabilität innerhalb einer Produktfamilie ermöglicht. Allerdings sind bestehende SPLE-Tools oft komplex und kostspielig, sodass sie für den Bildungsbereich unzugänglich sind. Diese Arbeit befasst sich mit dieser Lücke, indem sie die Integration von Product Line Engineering-Funktionen in die Open-Source-Bildungsplattform CADdrive konzipiert.

Die Forschung untersucht, wie zentrale SPLE-Prinzipien – wie Variabilitätsmodellierung und die Verwendung eines 150 %-Modells – für die Konfiguration physischer Produkte unter Verwendung von digitalem LEGO als Low-Fidelity-Tool angepasst werden können. Eine strukturierte Methodik umfasst eine Literaturrecherche, eine Fallstudie zu einer Drohnen-Produktlinie und einen Prototyp einer Benutzeroberfläche innerhalb von CADdrive. Die zentralen Forschungsfragen untersuchen die Anpassung von SPLE-Techniken für digitales LEGO, was zur Erstellung eines Merkmalsmodells, eines 150 %-LEGO-Modells und eines Algorithmus zur automatischen Ableitung von Produktvarianten führt. Eine wichtige Innovation ist die Methode, das grafische Merkmalsmodell mit dem 3D-Modell zu kombinieren, indem es als Formel ausgedrückt wird, die in die LDraw-Dateistruktur eingebettet ist.

Der vorgeschlagene konzeptionelle Rahmen zeigt, dass CADdrive erfolgreich zu einem praktischen Lehrmittel für das Produktlinien-Engineering ausgebaut werden kann. Es bietet Benutzern eine praxisorientierte Umgebung, in der sie lernen können, wie man Produktvarianten entwirft, verwaltet und ableitet, und schließt damit die Lücke zwischen theoretischen SPLE-Konzepten und ihrer Anwendung in der physischen Produktentwicklung.

CONTENTS

1 Introduction	1
1.1 Context	1
1.2 Problem Statement	2
1.3 Research Objectives	2
1.4 Research Questions	2
1.5 Research Methodology	2
1.6 Outline Of The Thesis	3
2 Theoretical Background	5
2.1 Product Line Engineering	5
2.1.1 Principles And Methodologies	6
2.1.1.1 Domain Analysis	7
2.1.1.2 Variability Modelling Techniques	8
2.1.1.3 Domain Implementation	12
2.1.1.4 Application Engineering	13
2.1.2 Product Line Adoption And Planning Approaches	14
2.1.2.1 Product Line Adoptions Approaches	14
2.1.2.2 Comparison Of Product Line Adoption Approaches	15
2.1.2.3 Product Line Planning Approaches	16
2.1.3 Examples Of Product Line Adoption In Industry	16
2.1.3.1 Product Line Engineering At General Motors	16
2.1.3.2 Product Line Engineering At Bombardier Transportation	17
2.1.3.3 Product Line Engineering At Raytheon	18
2.1.3.4 Challenges And Limitations	19
2.2 Product Architecture And Modularity	19
2.2.1 Module Interfaces And Constraints	21
2.2.2 Case Studies On Geometric-Aware Modeling	24
2.3 Caddrive Description	24
2.3.1 Caddrive Features	25
2.3.2 Caddrive's Formats	26
2.3.3 Caddrive In Education: Agile Product Development Case Studies	27
2.3.4 Caddrive's Limitations	29

3	Related Work.....	30
3.1	Lego Bricks In Education.....	30
3.1.1	Examples Of Using Physical Lego-Bricks In Educational Activities.....	30
3.1.1.1	Lego For Product Design And Manufacturing Simulations.....	30
3.1.1.2	Lego In Software Engineering Education	31
3.1.2	Examples Of Using Digital Lego-Bricks In Educational Activities	33
3.1.2.1	Istanbul Technical University, Turkey.....	33
3.1.2.2	Kinderuni Oö.....	33
3.1.3	Comparison Of Digital And Physical Lego	35
4	Fundamental Approach	37
4.1	Case Study Description.....	37
4.2	Atomic Modules.....	38
4.3	Module Interfaces.....	43
4.4	Variability Modeling.....	46
4.5	150% Drone Model.....	49
4.6	Mapping The Variability Model With The 150% Model	50
4.7	An Algorithm Of Deriving A 3d Model For A Selected Configuration	54
5	User Interface Prototyping	55
5.1	Iteration 1: Ultralight Drone.....	55
5.2	Iteration 2: Freestyle Drone	58
5.3	Iteration 3: Long-Range Drone	62
5.4	Mock-Up Ui Feature Model Tab.....	65
5.5	Mock-Up Ui Configuration Space	66
5.6	Mock-Up Ui Versions Tab.....	69
6	Evaluation, Discussion And Limitations.....	72
6.1	Case Study Evaluation	72
6.1.1	Positive Feedback.....	72
6.1.2	Criticism And Refinements.....	73
6.2	Ui Concept Evaluation	76
6.3	Answers To Research Questions.....	77
6.3.1	Research Question 1.....	77
6.3.1.1	Research Question 1.1.....	78
6.3.1.2	Research Question 1.2.....	78
6.3.1.3	Research Question 1.3.....	79

6.3.2 Research Question 2.....	79
6.4 Limitations	79
6.4.1 Reduced Utilisation Of The Sple Framework	80
6.4.2 Lack Of Formilized Guidelines.....	80
6.4.3 Lack Of Statistical Or Comparative Data	80
6.4.4 Only Conceptual Evaluation	80
6.4.5 Abstraction From Small-Scale And Repetitive Components.....	80
7 Conclusion And Future Research.....	82
7.1 Summary	82
7.2 Future Research.....	82
8 References	84

1 INTRODUCTION

1.1 CONTEXT

With today's shift from a seller's to a buyer's market, there is a clear trend towards product customization. Customers are constantly seeking products that cater to their specific needs and budgets. However, creating customer-specific designs can often be inefficient and costly, making it unaffordable for the average consumer. Besides, in most industries very few or practically no systems are created "from scratch", so engineers are likely to reuse knowledge from a previous project or product in the form of documents, processes, or models (Góngora et al., 2015).

To find a balance between reuse and customization in software and systems development, software product line engineering (SPLE) has emerged as a viable solution. SPLE is an approach to cost efficiently deriving tailored products for markets and customers, utilizing common components and services in a planned manner (Runeson & Engström, 2012). The focus is shifted from building isolated products to building families of related products, while reuse is discussed not at an individual object level, e.g. libraries or components, but as a whole: organizational, process-wise, and also lifecycle-wise end-to-end from requirements to actually deployed variability at the customer (Schwanninger & Kircher, 2009). This approach involves the development of a core platform as well as reusable components that can be easily customized to meet the specific needs of different products within a product line.

Originating in software development, SPLE principles have now spread far beyond the software domain. Companies in many other industries, such as automotive, aerospace, and consumer electronics have already integrated product line engineering solutions into their development processes to take advantage of reusability and variability management while reducing time-to-market, improving efficiency, and increasing customer satisfaction.

1.2 PROBLEM STATEMENT

Even though numerous commercial tools for product line engineering are available, there is currently no open-source equivalent. This paper seeks to develop such tool on the base of an open-source educational platform CadDrive. CadDrive aims to help students and employees to understand the entire engineering process – from requirements engineering to delivering the final product, and acquire skills in collaborative design and agile methodology. It has a wide range of functions from classic computer-aided design (CAD) and product lifecycle management (PLM) systems. Yet, it does not have a tool to support product line engineers.

1.3 RESEARCH OBJECTIVES

The primary objective of this research is to develop a concept of a product line engineering tool for CadDrive that leverages SPLE principles with a particular emphasis on creating geometric representations of product lines.

1.4 RESEARCH QUESTIONS

RQ1: How could SPLE techniques be adapted for digital LEGO?

RQ1.1: How could a variability model look like?

RQ1.2: How could a LEGO-based 150% model look like?

RQ1.3: How to derive a 3D model of a configuration from the 150% model automatically?

RQ2: How to integrate the framework into CadDrive?

1.5 RESEARCH METHODOLOGY

To achieve the research objectives and address the questions as mentioned earlier, a structured methodology is employed, comprising the following steps:

Literature Review: A thorough review of existing literature is conducted to explore current technologies and industry implementations related to SPLE, variant management, and modularity. This is built on foundational works such as those (Ulrich, 1995) (Kästner & Apel, 2013) which provide architectural foundation for SPLE and implementation techniques for variability. It also involves defining two core processes such as domain engineering and application engineering as well as variability modelling.

Case study development: To demonstrate how SPLE principles could be applied to physical products a drone use case is developed. It includes a drone variability model as well as a superset model comprising 3D models of drone modules in LDraw format.

Prototyping: A conceptual framework is designed to integrate SPLE principles into CadDrive, with a focus on physical products. Based on the framework a clickable PowerPoint prototype is modeled.

Validation: The prototype usability and functionality is tested by configuring three drone variants. Success is measured by CADdrive expert's confirmation of its practical feasibility.

1.6 OUTLINE OF THE THESIS

The thesis starts with an Introduction that establishes the context of the research and introduces the primary objective and research questions, setting the stage for the subsequent chapters.

Following the introduction, a Theoretical Background delves into foundational concepts of SPLE and product architecture, and then explains CADdrive's motivation and features. It based on foundational works such as those of (Ulrich, 1995) and (Kästner & Apel, 2013).

The Related Work chapter discusses how physical and digital LEGO is being used for teaching different concepts of software and product development.

The Fundamental Approach chapter demonstrates the structured approach used to answer the research questions. It explains the case study development and how a drone use case is utilized to illustrate how SPLE principles can be applied to physical products. This includes the creation of a drone variability model together with a 150% LEGO model and a variant derivation.

The User Interface Prototyping chapter describes the prototyping phase, where a conceptual framework is designed.

The Evaluation, Discussion and Limitations chapter explains how the case study and the user interface prototype are tested out and presents the findings from the case study and prototype validation. It addresses each research question systematically and discusses the work, highlighting limitations and suggesting potential improvements.

Finally, the Conclusion and Future Work chapter summarizes the research contributions, emphasizing the development of an SPLE-based tool for CadDrive and its applicability to geometric product line modeling.

2 THEORETICAL BACKGROUND

2.1 PRODUCT LINE ENGINEERING

PLE is a methodology for developing and maintaining a family of similar products (variants) that vary from each other in features and functions, taking advantage of the products' similarities while managing their differences. As illustrated in Figure 1 (Lameh et al., 2023), PLE stands in contrast to classical product-centric development, in which each individual product is developed and evolved independently from other products, or (at best) starts out as a cloned copy of a similar product that is then changed to suit the new product's specific needs (Krueger & Clements, 2013). Instead, it involves the development of a core platform and reusable components that can be easily customized enabling to maximize efficiency by reusing assets across mechanical, electrical, and software domains as much as possible.

Companies in different industries, such as automotive, aerospace, and consumer electronics are taking advantage of reusability while reducing time-to-market, improving efficiency, and increasing customer satisfaction. For instance, Lockheed Martin estimates that because of its SPLE-inspired approach it saves over \$47 million a year (S. Gregg et al., 2015), while Hewlett-Packard reports that it builds products 10 times as complex, with 1/4 of the staff, in 1/3 of the time, and with 1/25 of the bugs of previous products (Mebane & Ohta, 2007).

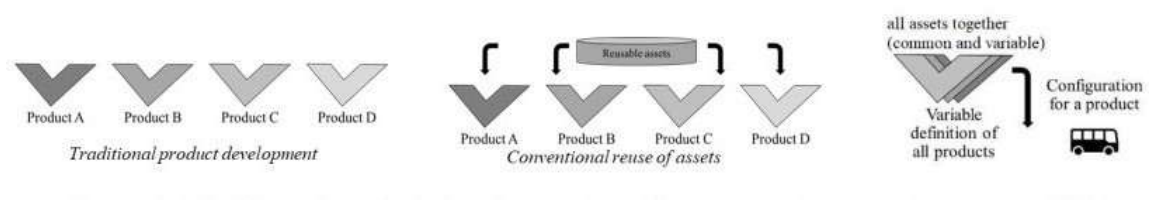


Figure 1: Traditional product development vs Conventional reuse of assets vs PLE (Lameh et al., 2023)

The origins of PLE can be roughly traced back to the mid 1970's with the appearance of the first works on the formalization of software product families (Góngora et al., 2015). Over the years, there have been increasing enhancements in the field. Most recently, the concepts have evolved to encompass systems and software engineering in a comprehensive way, as demonstrated by the development of industry standard families such as ISO 265xx focusing on practices for product lines. Hence, in software and product development PLE is extended to Systems and software product line engineering and refers to the production of a family of products that share a set of assets and common production facilities (Góngora et al., 2015).

2.1.1 PRINCIPLES AND METHODOLOGIES

A product line can be seen as a set of individual products that share a common, managed set of features addressing the specific needs of particular market segments or missions and that are derived from a common set of core assets in a systematic way (Clements & Northrop, 2002). In this context, a feature denotes a characteristic of a member product in a product line that distinguishes it from other member products in the product line (ISO, 2015). (Linden et al., 2007) formulated several principles that are fundamental in PLE including, among others, the principle of the two-lifecycle approach, which comprises the domain engineering and the application engineering lifecycle, and the principle of variability management. As illustrated in Figure 2, domain engineering and application engineering define a process of deriving a single variant while considering customer needs and domain knowledge. The framework also includes two perspectives: problem space, where common and variable requirements should be defined, and solution space, where a variant is derived from the core asset base. Moreover, there is connection (mapping) between features in the problem space and assets in the solution space, which describes which asset (artifact) belongs to which feature (Kästner & Apel, 2013).

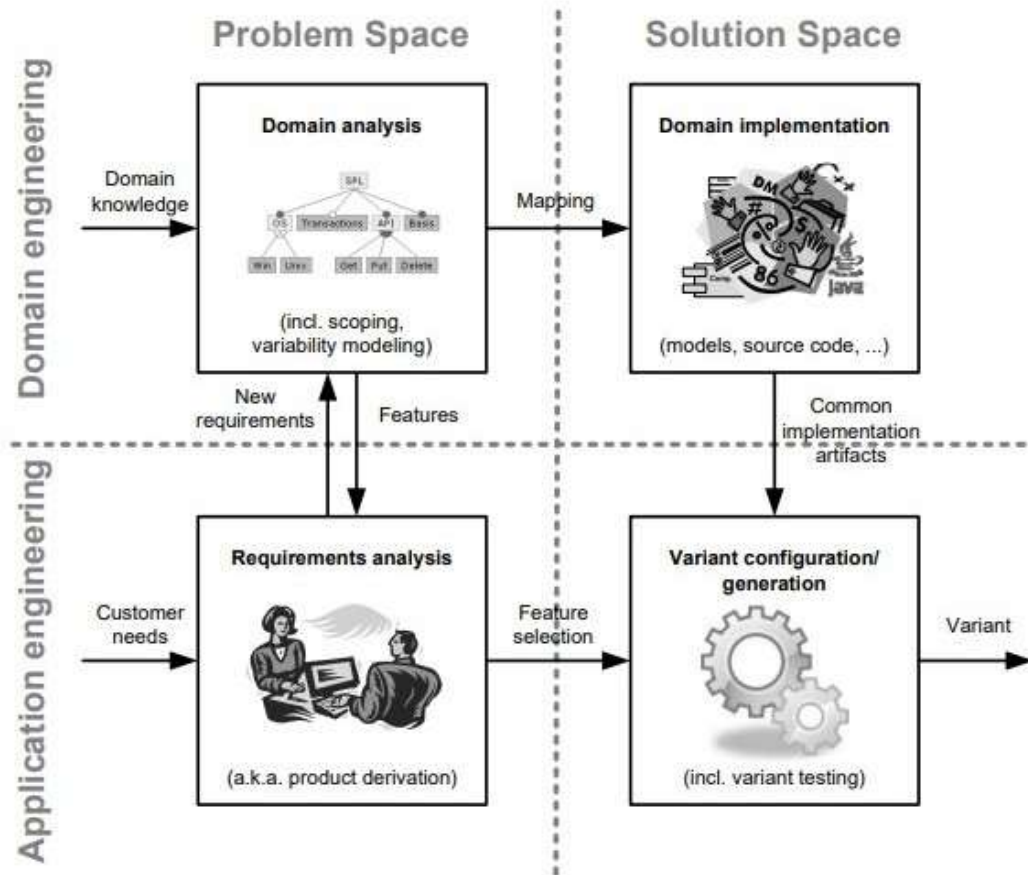


Figure 2: An overview of domain engineering and application engineering (Kästner & Apel, 2013)

2.1.1.1 DOMAIN ANALYSIS

To analyze the requirements of all current customers as well as anticipate requirements of potential future customers, developers should analyze all data related to a product, which is done in domain engineering. Domain engineering is a reuse-based approach defining the scope (i.e., domain definition), specifying the structure (i.e., domain architecture), and building the assets for a class of systems, subsystems, or member products (ISO, 2015). During domain analysis, developers determine the scope of the product line and identify its common and variable features, which they then document in a variability model. From these commonalities and differences, different variants that fulfilled the requirements of all the customers are derived. These variants form a product line.

2.1.1.2 VARIABILITY MODELLING TECHNIQUES

A number of techniques have been developed to manage variability, which all have their benefits and drawbacks. As (Romano et al., 2022) highlight, most approaches are either based on the Feature-Oriented Domain Analysis (FODA) approach by (Kang et al., 1990) (feature modeling) or on the Synthesis method [12] (decision modeling) [13]. There is also an approach which combines aspects of feature modeling and decision modeling, called Orthogonal Variability Model (OVM) (Pohl et al., 2005). Despite multiple attempts to unify variability modeling approaches, no standard variability modeling approach exists.

Feature Models

Feature models, originating from the FODA method, represent variability through hierarchical trees of features with parent-child relationships. They excel in visualizing product line commonality and variability using intuitive notations like mandatory, optional, and alternative features. Constraints (e.g., "requires" or "excludes") can be added to enforce dependencies.

Graphical representation of a Feature-diagram for an online shop is shown in Figure 3. It is organized as a hierarchical structure with key components like Payment, ProductSelection, UserManagement, and Design. The model breaks down into specific payment methods (DebitCard and CreditCard), user features (Newsletter, ShoppingBasket), and platform options (Mobile, Tablet, PC). The model includes several constraints that define relationships between features: Search requires Security, while Security and Payments have a mutual exclusion relationship. The Responsive design feature automatically includes Mobile, Tablet and PC options. The review system offers combinations of Stars, Numerical ratings and Comments. It shows both mandatory elements (like the Catalog) and optional features (like Wishlist), with clear logical relationships between components. (Feichtinger et al., 2022)

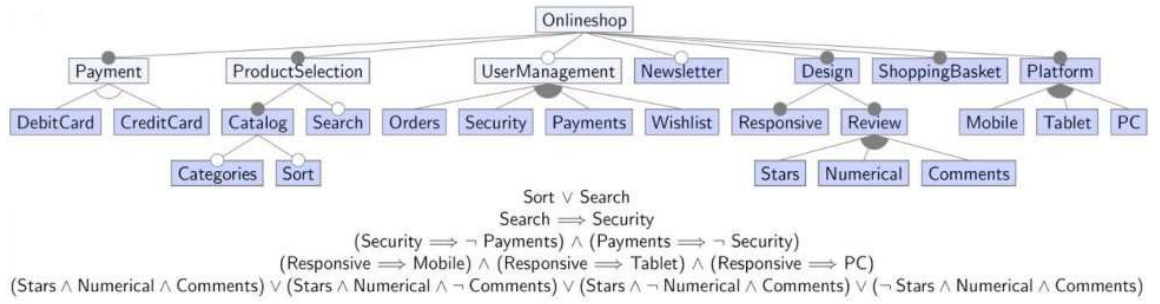


Figure 3: Feature-diagram example of an online shop (Feichtinger et al., 2022)

Such models excel at capturing commonalities and variability explicitly and support complex constraints through propositional logic. However, they lack native support for numerical or string values during configuration, unlike decision models, and have limited expressiveness for dynamic rules.

Decision models

Decision models take advantage of existing commonalities and focus only on varying features and their constraints (Campbell et al., 1990). They focus on capturing variability through explicit configuration decisions rather than hierarchical features. Unlike feature models, decision models separate variability specification from system models, enabling better integration with engineering workflows. However, they can become overly complex for non-technical stakeholders due to their procedural nature and lack of visual intuitiveness compared to feature trees.

Graphical representation of a decision model for the online shop is shown in Figure 4. It is structured as a table with key decision points and their properties. The model includes various decision types such as Boolean choices (Search, Categories, Sort), Enumerations (Payment, UserManagement, Review), Numerical values (Newsletter_percent), and String inputs (Newsletter_mail). The model defines constraints and rules that govern the relationships between decisions. For instance, selecting Search automatically sets UserManagement to "Security," while choosing Sort enables Search. The UserManagement options have mutual exclusion rules (Security disables Payments, and

vice versa). The Review system allows combinations of Stars, Numerical ratings, and Comments with a cardinality of 2:3. (Feichtinger et al., 2022)

ID	Question	Type	Range	Card.	Constraint/Rule	Visibility
Payment	Which payment methods should be supported?	Enum	DebitCard CreditCard	1:1		
Search	Should a search function be supported?	Boolean	true false		if (Search) { UserManagement = Security }	
Categories	Split products into categories?	Boolean	true false		if (!Search) { Sort = true }	
Sort	Should products be sortable?	Boolean	true false		if (!Sort) { Search = true }	
UserManagement	Which user management options should be added?	Enum	Orders Security Payments Wishlist	1:4	if (Security) { disAllow(Payments) }	
Newsletter_percent	How many percent off?	Number	0 - 100		if (!Security) { allow(Payments) }	
Newsletter_mail	From which mail should the newsletter be sent?	String			if (Payments) { disAllow(Security) }	
Review	Which review techniques should be supported	Enum	Stars Numerical Comments	2:3	if (!Payments) { allow(Security) }	

Figure 4: Decision model example of an online shop (Feichtinger et al., 2022)

The strengths of decisions models lie in dynamic configurability, such as numerical inputs and string-based decisions, and the ability to define complex dependencies through rules. However, they do not inherently support hierarchical structures like feature models, and transformations to feature models lose abstract or hidden properties.

Orthogonal variability model (OVM)

OVM combine aspects of feature modeling and decision modeling. Variability is described through variation points (what can vary) and variants (how it can vary). A variation point is the specific place in a PL artifact to which a design decision is connected (Edson et al., 2005). Each variation point is associated with a set of variants that corresponds to design alternatives to resolve the variability (Trigaux & Heymans, 2003). In addition, constraints and dependencies are added to the variability model. Constraints shows which combinations of features are valid within a product variant, while dependencies specify relationships between features, indicating that one feature's selection affects another.

Graphical representation of an OVM for the online shop is shown in Figure 5. The model identifies core variable elements of the system through variation points, each representing a distinct configuration option. Key variation points include payment methods (Debit Card, Credit Card), product organization (Categories, Product Selection, Catalog), user interface features (Sort or Search, Search, Sort), user management components (Orders, Security, Payments, Wishlist), and review system options (Stars, Numerical, Comments).

The model follows OVM's characteristic separation of variation points from variants, presenting a modular view of the system's variability. (Feichtinger et al., 2022)

Unlike the hierarchical structure of feature models or the rule-based approach of decision models, this representation focuses on orthogonal variability - where variation points can be independently selected within defined constraints.

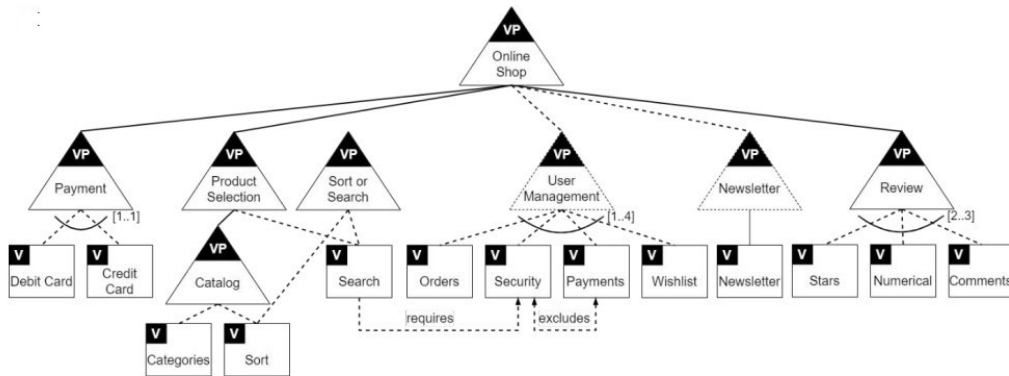


Figure 5: OVM of an online shop (Feichtinger et al., 2022)

Best variability model for educational purposes

When teaching mechanical engineers about variability modeling for applications like product design, manufacturing systems, or modular robotics, the optimal modeling approach depends on specific educational objectives and complexity levels.

For introductory courses, feature models offer the most accessible entry point due to their simple hierarchical structure of features with optional and mandatory choices. Their intuitive nature makes them ideal for teaching fundamental concepts of variability, and their widespread academic use ensures ample learning resources. However, these models lack capabilities for dynamic rule implementation or numerical parameter handling, limiting them to static configurations like basic product variants.

For more advanced applications, decision models provide greater practical utility by supporting numerical and string parameters essential for mechanical engineering

scenarios, along with dynamic rules that can modify configurations based on parameter values. This makes them particularly valuable for teaching design automation and smart manufacturing concepts, though their steeper learning curve and fewer educational resources present challenges for beginners.

OVMs serve niche applications, particularly for graduate-level discussions of complex modular systems. While useful for analyzing independent variability points in sophisticated machinery configurations, their less intuitive structure and absence of numerical rule support reduce their effectiveness for most educational settings compared to the other approaches. The choice among these models should align with both the students' experience levels and the intended learning outcomes of the course or training program.

2.1.1.3 DOMAIN IMPLEMENTATION

The digital representations of the individual products in a product line can include, but are not limited to, requirement and design specifications, system and computer-aided design (CAD) models, source code and build files, test plans and cases, user documentation, repair manuals and installation guides, project budgets, schedules and work plans, product calibration and configuration files, data models, parts lists, and more (Clements, 2019). A wide-spread technique for the digital representation of product lines and their product variants are so-called 150% models (Clements, 2019). Such models combine all relevant features and characteristics of all product variants of the product line into a common superset model. Furthermore, such models include all possible configurations of features and characteristics for all the variants and, hence, provide a complete picture of the product line.

Figure 6 illustrates a drone's 150% model which represents all the possible combinations of features for all the possible variants. For example, this simplified architecture contains three different tank sizes (a little tank, a medium tank, and a huge tank) even though any finished UAV will only have one tank.

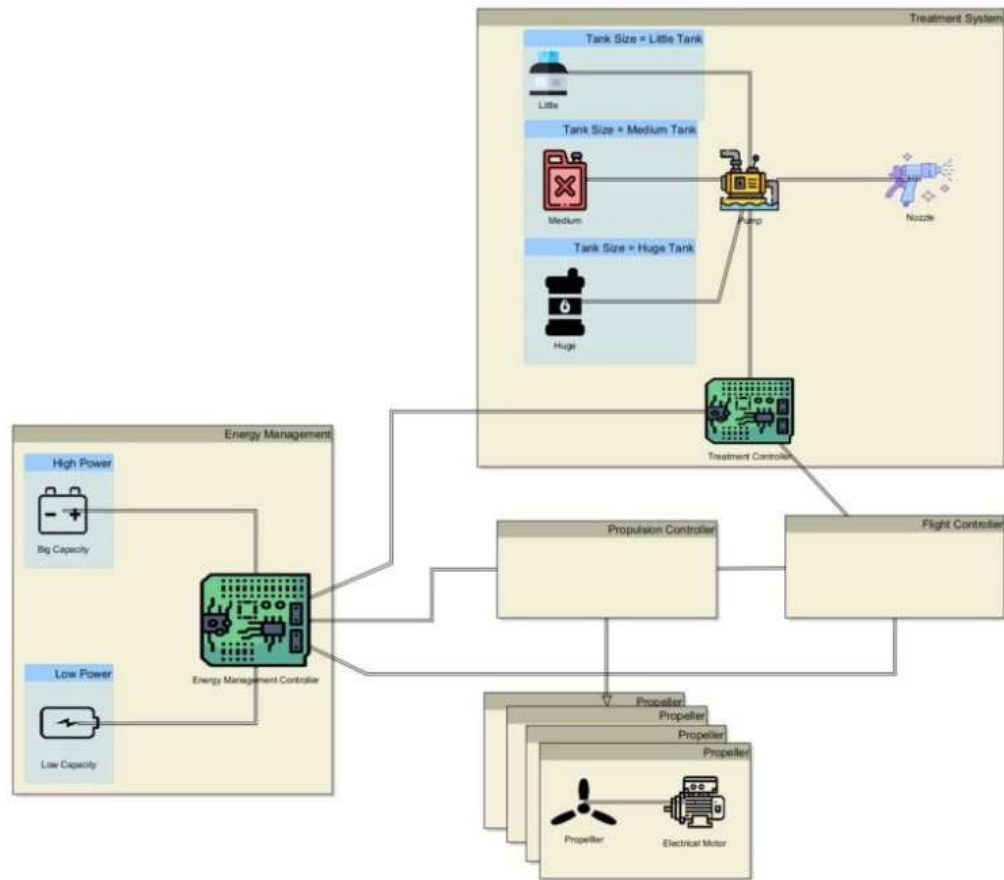


Figure 6: A simplified 150% model of an UAV drone (Dube, 2021)

2.1.1.4 APPLICATION ENGINEERING

In contrast to domain engineering, application engineering represents the process of deriving a single product variant that is tailored to the requirements of a specific customer, based on the results of domain engineering (Kästner & Apel, 2013). That means that a configuration of a particular product of the product line is created by narrowing down the assets according to the feature choices of that particular product. However, with the appearance of second-generation PLE, the importance of the application engineering process steadily decreases, while products are derived through the use of high-end industrial-strength automation that configures the shared assets appropriately for each product (Krueger & Clements, 2013).

The tools within that the process is performed must be able to operate with all the models and documents related to a product. Several commercial tools such as Gears2 and pure::variants are widely used for performing the process. They can integrate with Model-Based Systems Engineering (MBSE), DevOps, IDE tools, PLM & Enterprise Systems and so on. There are also open-source alternatives for modelling variability like FeatureIDE, KConfig or VaryLaTeX, though they are more for academic or small-scale projects.

2.1.2 PRODUCT LINE ADOPTION AND PLANNING APPROACHES

(Schmid & Verlage, 2002) identify several different types of situations for adopting PLE and different product line planning techniques.

2.1.2.1 PRODUCT LINE ADOPTIONS APPROACHES

According to (Schmid & Verlage, 2002) there are four product line adoption schemes for transitioning to product line engineering, including independent, project-integrating, reengineering-driven, and leveraged.

Independent adoption approach

Using the independent adoption approach, which is rather uncommon, a company starts a new product line from scratch with no existing products. This allows full control over architecture but requires significant upfront investment. Detailed analysis of potential portfolios, market analysis, feasibility studies are necessary. However, even if all of them are performed, the company still faces significant uncertainty. (Schmid & Verlage, 2002)

Project-integrating adoption

With project-integrating adoption, multiple existing projects are merged into a unified product line. These products are developed separately, but have some commonalities, which are later generalized into common, reusable components. This is incremental but companies must expect more problems with the degradation of the reference architecture within time. (Schmid & Verlage, 2002)

Reengineering-driven adoption

A company usually adopts this approach if when it finds that its software development faces significant obstacles. For example, if the cost of product development grows too high or if it becomes impossible to derive new products based on the available systems. With reengineering-driven adoption, legacy systems are restructured into a product line, often due to high maintenance costs. This can be done via a "big bang" (full rework) or incrementally. (Schmid & Verlage, 2002)

Leveraged adoption

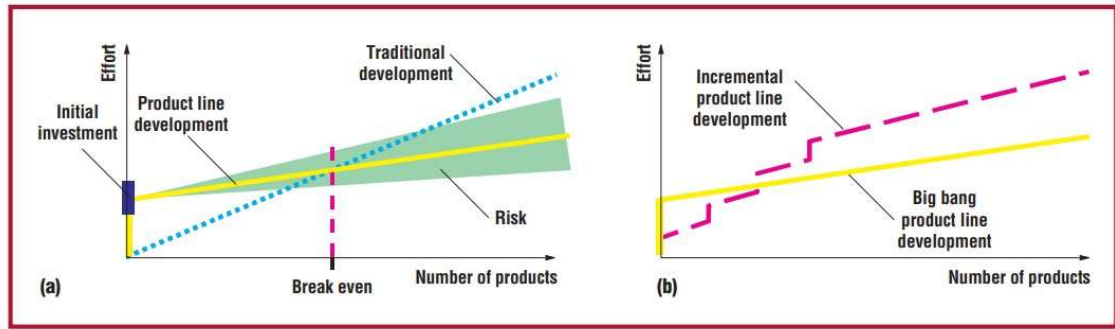
This approach requires an existing product line and is characterized by a shift to a new market. This reduces risk by reusing proven infrastructure. A company has a competitive advantage right from the start entering new market segment with low costs and few risks. (Schmid & Verlage, 2002)

2.1.2.2 COMPARISON OF PRODUCT LINE ADOPTION APPROACHES

Figure 7 illustrates two key product line adoption strategies (the big bang and incremental) and their economic impacts through investment curves. The big bang method requires a substantial initial investment to establish a product line infrastructure before launching any products. While it has a slower start, it eventually outperforms traditional development in cost efficiency and scalability as the number of products grows.

The incremental approach starts with minimal reusable assets and expanding them step-by-step alongside product releases. It requires lower investment costs, generates revenue earlier and reduces risk by allowing continuous adjustments based on market feedback. The full efficiency benefits of reuse take longer to materialize compared to the Big Bang approach.

The big bang approach is ideal for stable markets with predictable requirements, while the incremental approach is better suited for dynamic environments or organizations with limited capital.



Product line investment curves: (a) the big bang approach, including risks; (b) the big bang versus incremental approach.

Figure 7: Comparison of product line adoption approaches (Schmid & Verlage, 2002)

2.1.2.3 PRODUCT LINE PLANNING APPROACHES

Product line planning techniques focus on strategic decision-making through product portfolio scoping, domain-based scoping, and reuse infrastructure scoping. These techniques optimize investments and align adoption with business goals.

During product portfolio scoping a company defines which products will be part of the product line based on market demand, competition, and feasibility. This helps avoid redundant or conflicting products. (Schmid & Verlage, 2002)

During domain-based scoping a company identifies which technical areas (e.g., security, UI, data processing) offer the highest reuse potential, ensuring efficient resource allocation. (Schmid & Verlage, 2002)

During reuse infrastructure scoping it determines which functionalities should be built as reusable assets (e.g., shared components, configurable modules) to minimize redundant development. (Schmid & Verlage, 2002)

2.1.3 EXAMPLES OF PRODUCT LINE ADOPTION IN INDUSTRY

2.1.3.1 Product line engineering at General Motors

(Flores et al., 2012) explain how Second-Generation Product Line Engineering helps to manage the extreme complexity of General Motors' vehicle electronics with the usage of

Gears (BigLever). Vehicles like the Chevy Volt contain over ten million lines of code, surpassing even a Boeing 787. Additionally, GM had to handle immense variability, with thousands of feature combinations. Ensuring traceability across requirements, design, code, and manufacturing further compounded the difficulties.

Gears (BigLever) introduces a feature-centric approach, replacing traditional bills-of-materials with a bill-of-features. Feature models defined constraints—for example, "Park Assist requires software-controlled brakes." The solution also incorporated hierarchical product lines, nesting production lines to align with GM's organizational structure (e.g., Safety → Subsystems → Components). Automation played a crucial role, with Gears integrating tools like IBM DOORS for requirements, Rhapsody for SysML modeling, and manufacturing systems. Variation points were injected into assets (requirements, code, calibrations), enabling automated product derivation.

As a result, GM managed to reduce complexity dramatically - from $O(n^2)$ to $O(n)$, and establish a reuse-driven approach as a single source of truth for 60+ vehicle models across seven brands.

2.1.3.2 Product line engineering at Bombardier Transportation

(Abbas et al., 2020) explore how Bombardier Transportation (BT) adopted SPLE to address the challenges of developing customized software for train propulsion control systems. Trains require software tailored to varying regional standards, hardware configurations, and performance requirements (e.g., speed, capacity). Traditional project-based development led to long cycles (6–12 months per product) and duplicated efforts.

To adopt SPLE BT created a 150% SPL with reusable Simulink components (software models) that were mapped to the Requirements Specification, which served as a features description document and is tailored for each product. Each component was tested individually with its own mini-test environment. For a new train model, existing components are integrated differently, adjusted, removed or copied (clone-and-own).

From the Simulink models C/C++ code is generated and tested on different level from testing small building blocks to testing the entire propulsion system.

BT chose an incremental adoption strategy for SPLE, avoiding high upfront costs. Existing projects were copied and adapted for new requirements, leveraging past work without a full domain engineering phase. As a result, SPLE reduced development time from 6 months to 1 month for new features and improved confidence in derived products.

2.1.3.3 Product line engineering at Raytheon

(Young & Clements, 2017) discuss the integration of Model-Based Engineering and PLE at Raytheon. While Model-Based Engineering uses formal models to improve analysis, automation, and early problem detection, PLE manages portfolios of related products by leveraging commonalities and automating variations. Raytheon applied these methodologies to its Integrated Air and Missile Defense system, GloboShield, using IBM Rhapsody for Model-Based Engineering and BigLever's Gears for PLE.

Gears is used to define a feature model which captures all possible variations in GloboShield as well as dependencies and constraints to ensure valid combinations. It also specifies different product configurations - feature profiles. In IBM Rhapsody (SysML), model elements (blocks, interfaces, behavioral diagrams) are marked as variation points using Gears annotations. These model elements appear or disappear based on feature selections. Gears' domain-specific logic then evaluates the feature profile and automatically includes or excludes the tagged elements, generating a product-specific instance of the architecture. The configured model highlights included components in yellow and excluded ones with a red slash, providing immediate visual feedback. After configuration, Gears stages the output by removing unused variations and metadata, resulting in a clean, product-specific model. (Young & Clements, 2017)

Raytheon follows a well-planned and incremental adoption of PLE that provides incremental return on incremental investment, so that at every stage the adoption is providing positive value (S. P. Gregg et al., 2020). The approach helped Raytheon to

reduce artifacts (80% fewer models to maintain), establish automated, error-free derivations of product architectures, and provide scalability and consistency across the product line. (Young & Clements, 2017)

2.1.3.4 Challenges and limitations

The examples above demonstrate that numerous industries use advanced tooling, which requires significant expertise and organizational maturity. While Bombardier Transportation relies on complex tools like Simulink and manual trimming of components to manage different train configurations, the approach of Raytheon to manage its missile defense product line combines PLE with Model Based Engineering, which also requires comprehensive tools like Gears and Rhapsody. These industrial tools are costly, complex, and burdened by restrictive licenses, making them impractical for education. The aim of this research is to demonstrate that CADdrive and digital LEGO, as lightweight alternative, provides an accessible way to teach core PLE principles without the overhead of enterprise software and enables students to focus on learning rather than tool complexity.

2.2 PRODUCT ARCHITECTURE AND MODULARITY

It is worth mentioning that PLE often goes hand in hand with the concept of modularity. Modularity enables the configuration of multiple end products from a limited number of modules allowing customers to mix and match elements to come up with a final product that suits their tastes and needs (Baldwin & Clark, 2003). It also enables design flexibility over the life of a product, which can correspond to changes in functional requirements over time. PLE on the other hand manages and optimizes a company's product diversity in order to control complexity and to balance out too many variants and versions.

Modularity is an essential characteristic of product architecture. (Ulrich, 1995) formulated three technical decisions that together form the product architecture. The first is the arrangement of functions, which describe what a product does, by linking their inputs and outputs (of energy, material, or information). The second is the mapping from functions

to physical building blocks. It demonstrates which building blocks are connected to which function or functions. The third is the specification of physical interfaces among interacting building blocks to enable the physical realization of exchanges of energy, material, and information, and/or their geometric connection. (Ulrich, 1995) states that modular product has a one-to-one mapping between functions and building blocks, and its interfaces are de-coupled. It means that each module delivers a specific function of a product or system and can be designed independently from other modules. To achieve the research objectives and address the questions as mentioned earlier, a structured methodology is employed, comprising the steps, which are shown in Figure 8.

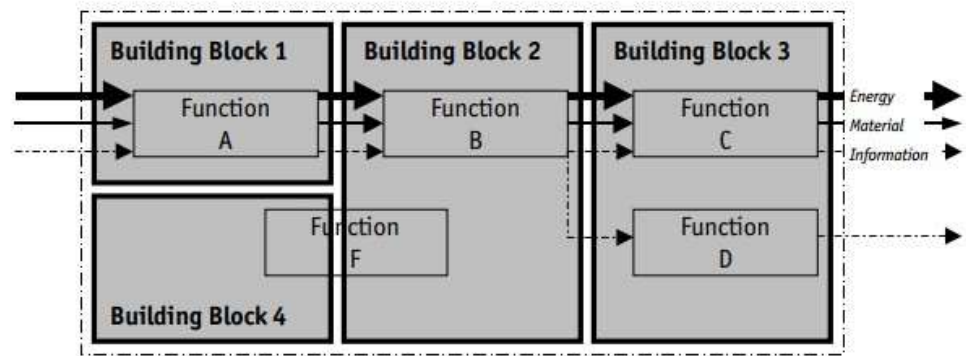


Figure 8: A conceptual illustration of product architecture (Ulrich, 1995)

While Ulrich defines modularity in terms of product architecture, (Piran et al., 2016) extend the concept to another contexts, such as design modularity, modularity in production, modularity in use, organizational modularity, service modularity and environmental modularity. In this research, modularity is defined as a systematic approach where a product or system is composed of various modules, and these modules can be combined in different ways to form different products with individual characteristics (Li et al., 2019).

Clear rules should be defined to reassure that all subsystems will be compatible and function correctly in the final design (Baldwin & Clark, 2003). Such rules concern, among other things, the syntactic and semantic module interfaces, which define how the modules can be connected and interact. The effects of a misspecified interface can go beyond module incompatibility to unexpected product behavior (J. Parslov & Mortensen, 2015).

2.2.1 MODULE INTERFACES AND CONSTRAINTS

When applying PLE principles to the development of physical products, it is important to consider the physical nature of module interfaces. In this research, interfaces are defined as physical connections (e.g., LEGO blocks snapping together) and are parts of system elements, though there is no consistent definition or perception of what is a "product interface." According to (J. F. Parslov & Mortensen, 2015) definitions vary widely in how interfaces manifest – whether they are physical connections (e.g. how two LEGO blocks snap together), or functional connections (e.g. how electricity flows between parts), or abstract relationships (e.g. how software components should communicate). Definitions also differ when it comes to whether an interface is a part of system elements (e.g., the plug on a charger is part of the charger itself) or a separate design object (e.g., a USB port is a separate thing that both the charger and phone must match). The language to describe interfaces also lacks consistency – some people use structural terms (e.g., the bolt fits into the nut), other use functional (e.g., the motor transfers power to the wheel) or systems terms (e.g., the module exchanges data with the subsystem).

This narrower focus aligns with Ulrich's emphasis on mechanical, contact-based interactions between components. Such contact-based interactions usually entail constraints on the contact points of the mechanical surfaces to guarantee the required function of the assembly. For physical products, these interfaces require precise geometric constraints, modeled in computer-aided design (CAD) to guarantee assemblability. By specifying these constraints in CAD models, designers can ensure the proper arrangement and compatibility of parts within the products of the product line.

Figure 9 categorizes geometric constraints used in assembly modeling to define relationships between components in CAD systems. These constraints ensure that parts fit together correctly and maintain intended functionality during design changes. (Wang, 2009)

Each constraint serves a specific purpose in maintaining design integrity. Coincident constraints force two geometric elements to occupy the same space, such as aligning part corners, edges, or surfaces. Parallel constraints ensure lines or planes remain parallel,

commonly used in sliding mechanisms, while perpendicular constraints maintain 90° angles between features, crucial for bracket supports and cross beams. Vertical and horizontal constraints align features to global axes, important for mounting plates. Other constraint types include on-entity constraints that fix points or curves to geometric entities, and symmetric constraints that mirror features across lines or planes. Equal constraints enforce identical dimensions for repeated elements like bolts or hole patterns, and distanced constraints maintain fixed gaps between components. Angular constraints lock specific angles between features, while aligned constraints center features relative to references. Against constraints prevent part penetration.

The constraints are particularly valuable for standardizing module connections and module variants, which is particularly valuable for both modular design and product line engineering. The practical application of these constraints can significantly improve design efficiency, particularly in systems requiring mix-and-match modularity like LEGO or configurable product platforms.

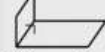
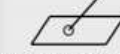
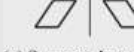
Type of constraints	Figures and description		
Coincident	 (a) co-point	 (b) co-edge	 (c) co-face
Parallel	 (a) line parallel to line	 (b) face parallel to line	 (c) face parallel to face
Perpendicular	 (a) line - line	 (b) face - line	 (c) face - face
Vertical			
Horizontal			
On-Entity	 (a) On the same point	 (b) Point on curve	 (c) Point on surface
Symmetric	 (a) Symmetric to a line	 (b) Symmetric to a face	
Equal	 (a) Equal Length	 (b) Equal radius	
Distanced	 With constant distance to reference entity		
Angular	 With constant angle to a line.		
Aligned	 N0		
Against	 N0		
Mated			

Figure 9: Geometric constraints in assembly modeling (Wang, 2009)

2.2.2 CASE STUDIES ON GEOMETRIC-AWARE MODELING

Modularity's practical implementation hinges on resolving geometric conflicts early. (Warniez et al., 2014) emphasize that early integration of geometric specifications is critical to avoid late-stage design conflicts and propose to formalize geometric constraints via a SysML profile. The profile aims to support geometrical metrics (e.g., dimensions, positioning) to assess physical integration of different physical architectures of mechatronic systems. It incorporates simplified geometric primitives (e.g., cylinders, parallelepipeds) and topological constraints to model component interfaces at the system level, enabling early evaluation of spatial integration. However, while this approach facilitates rapid architectural comparisons, it lacks a direct connection to physical prototyping, making it challenging to validate real-world assemblability.

(Koch et al., 2016) highlight the role of geometric constraints in ensuring compatibility and propose integrating CAD models with feature models to manage geometric dependencies. A modular Pick&Place Unit is used to illustrate the approach. Its variants depend on industry type (e.g., pharmaceutical) and work piece materials, with constraints on part arrangements (e.g., Stack and Crane must not overlap). Though the approach enables holistic product line specification, combining variability and geometry, it requires specialized CAD toolchains, limiting accessibility for education or early-stage design.

This research bridges this gap by using digital LEGO as a low-fidelity alternative for early spatial validation (like SysML) but with precise geometry (unlike abstract SysML volumes).

2.3 CADDRIVE DESCRIPTION

In today's competitive market, going from an idea to the market launch is a slow, risky and unsustainable process – not only for enterprises but for educational institutions and research organizations as well. CadDrive addresses this challenge as an open-source product development platform designed for schools, universities, and companies, fostering a collaborative ecosystem that accelerates innovation. By opening up the design process

to a community of developers and end-users, companies can gain quick access to innovation, prototyping and product testing.

For that reason, CadDrive learning platform can be used in companies for training employees to acquire new skills such as collaborative design and Agile methodology, particularly the scrum framework, and learn how to work as a team to solve a given problem and how to use digital tools to support the process. It can also be used as a self-learning tool for people with years of engineering experience who have an old way of working so that they can develop skills to function well in today's digital world. Moreover, CadDrive serves as a pedagogical instrument in academic settings, enabling educators to enhance student engagement in engineering disciplines by providing an interactive, project-based learning environment that mirrors real-world product development workflows. The platform enables students to understand the entire engineering process from an idea generation to delivery of the solution. Students can exercise in teams and learn how to distribute and prioritize task, who is doing what and who are reporting to.

The core motivation of the Caddrive team is to increase the number of young people who go to study STEM subjects.

2.3.1 CADDRIVE FEATURES

Public and private product design repositories: Users can manage public product design repositories that are visible to everybody worldwide and private product design repositories that are visible only to the people you select.

Design revisions with branching and merging: Users can track the process of a product evolving over the time with an adequate information system.

An integrated 3D viewer with a rich set of interaction possibilities: People around the world can look at design revisions from different perspectives.

Issue management with design revision integration: It lets practitioners track different opinions of different stakeholders and make a right design decisions.

Time management with team performance control: Users can plan and control a task execution across the entire product design project.

The platform displays projects in two tabs: Public for openly visible designs and Private for restricted-access projects. Each project shows its latest 3D model, name, description, version and issue counts, member list, and owner icon. The tab Version documents a project's development stages, from initial drafts to final models. Each version includes a code, name, assigned contributors, and an interactive 3D preview with a component tree for navigation. Tasks and assignments are tracked in Issues, with project leaders setting deadlines, assigning team members, and managing file uploads (PDFs, presentations) for review. In Milestones, a timeline view tracks task progress, showing open and closed issues per sprint. Deliverables, for example simulation results, can be downloaded for evaluation, while team discussions happen in real-time chat. In Members all collaborators are listed with project access. New members can be invited and assigned roles (e.g., engineer, manager). Settings are for adjusting project details like name, description, and visibility (public/private).

2.3.2 CADDRIVE'S FORMATS

Caddrive supports different types of formats to visualize CAD models, such as STEP, COLLADA and LDraw™. While STEP and COLLADA are supported by multiple CAD, CAM, and 3D modeling tools and widely used in engineering, LDraw™ serves as an open standard for LEGO CAD programs such as LeoCad and allows users to create virtual LEGO models and scenes. The LDraw data format represents digital LEGO models as hierarchical (i.e. tree) structures with reusable assemblies as root and internal nodes, and bricks as leaf nodes. Furthermore, the root node defines a world coordinate system, while the child nodes down to the leaf nodes define their local coordinate systems, which are relative to their parent coordinate systems. Consequently, the digital LEGO bricks (i.e. the leaf nodes) as well as assemblies (i.e. the internal nodes) can be positioned and oriented within the local coordinate system of their parent assembly node.

2.3.3 CADDRIVE IN EDUCATION: AGILE PRODUCT DEVELOPMENT CASE STUDIES

CADDrive was successfully used for teaching Innovation and Product Management students in 2024 and 2025. Agile engineering projects developed in CADDrive include a self-sufficient Glasshouse, a rotating house Golazo, and a sustainable toilet solution EcoFlush.

The Glasshouse (see Figure 10) was developed in 2024 when CADDrive was used by innovation students for the first time for product development. The purpose of the glasshouse is growing vegetables in Austria all year round. Thirteen students divided into different teams collaborated in the Scrum-based environment using tools such as retrospectives and Scrum dynamics. The final model is built of digital LEGO in LeoCAD.

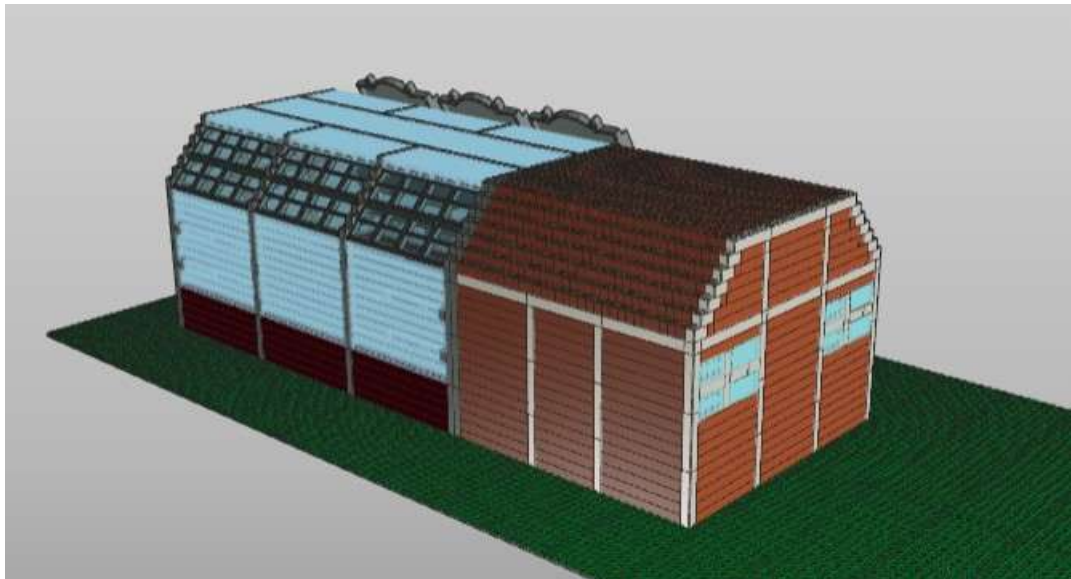


Figure 10: The Glasshouse (Hackenberg & Zehetner, 2025)

The Golazo and the EcoFlush were developed by innovation students in 2025. The aim of the projects was to design a sustainable greenhouse incorporated with a vertical farm.

The Ecoflush (see Figure 11) is a sustainable, self-sufficient public toilet concept that uses natural filtration, solar power, and smart glass technology. The toilet aims to address the lack of public restroom facilities and the need for sustainable, clean, and aesthetically integrated public sanitation solution.

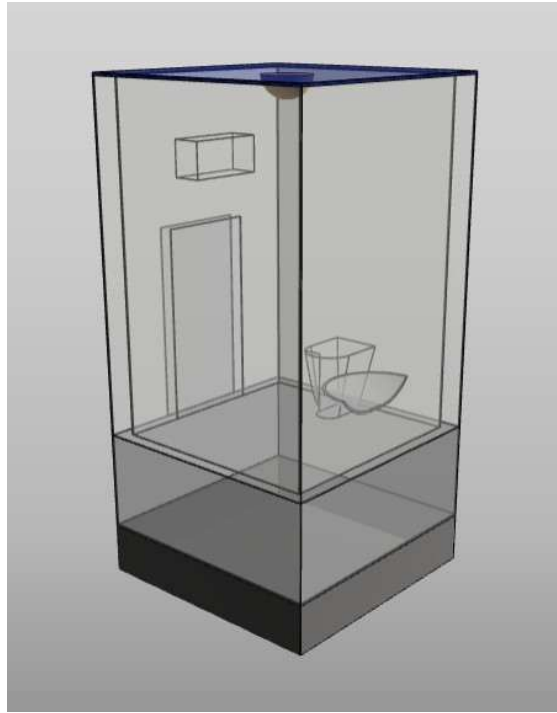


Figure 11: The EcoFlush (Hackenberg & Zehetner, 2025)

The Golazo (see Figure 12) is an energy efficient glasshouse incorporated with a vertical farm. The 3D model is developed in SolidWorks. Due to its cylindrical shape and an underground rotating mechanism, the building can follow the sun movement. While the plants on the walls of the building obtain the same amount of light during the day, which results in uniform growth and significant savings in lighting, the solar panels always get the right angles to maximize electricity generating.

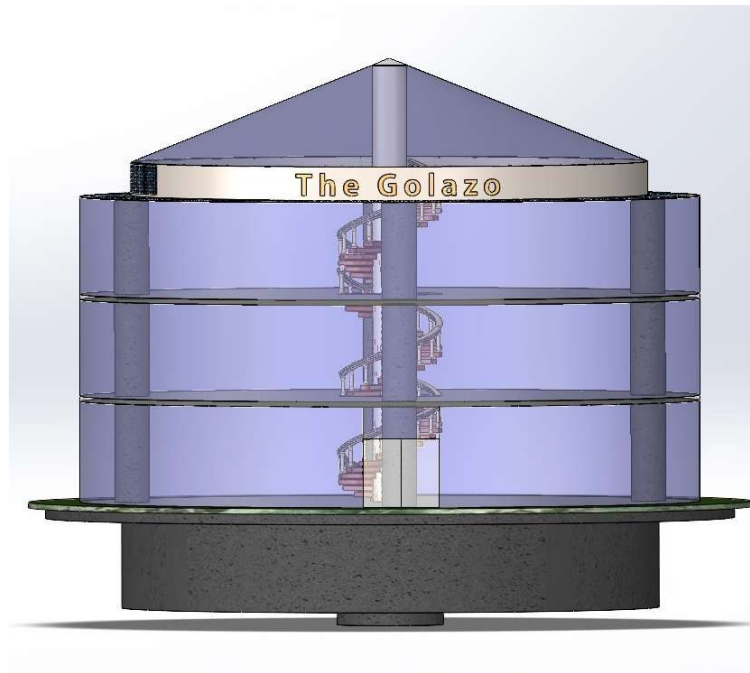


Figure 12: The Golazo (Hackenberg & Zehetner, 2025)

Working in teams of four and five participants and following Scrum framework and milestone-based approach, students went through different stages of product development such as planning and requirements definition, concept development, CAD modelling, testing, cost planning and marketing. They learnt not only the product development process but basics of project management and data management as well. All the projects succeeded in functional design and proved the effectiveness of Caddrive in academic environment.

2.3.4 CADDRIVE'S LIMITATIONS

Although CADDrive supports version control, collaboration and issue tracking, it lacks dedicated product line engineering features, which are critical for product development. These features could help students learn how to design scalable product lines and reuse components across them, as well as providing an understanding of platform-based development — a key strategy in industries such as automotive and consumer electronics. Furthermore, product line engineering fosters a modular approach, enhancing sustainability and profitability, and prepares students for mass customization trends.

3 RELATED WORK

3.1 LEGO BRICKS IN EDUCATION

The LEGO Group was founded in 1932 by Ole Kirk Kristiansen. The name ‘LEGO’ is an abbreviation of the two Danish words “leg godt”, meaning “play well”. The company has since passed from generation to generation and is to this day still owned by the Kirk Kristiansen family.

Since the beginning, LEGO bricks have been a popular playing activity among people of all genders and ages. The use of LEGO toys stimulates creative cognition and promotes non-traditional problem-solving strategies through interactive construction.

Traditional learning approaches like master classes are useful to teach theoretical concepts, but they do not seem to be very appealing to today’s students. Therefore, it is important to complement the traditional methods with more innovative and active learning methods. (Lopez-Fernandez et al., 2021). These methods affect positively on the teaching-learning processes because they enhance the student’s motivation, facilitate the acquisition of technical knowledge, and contribute to the development of the much-demanded soft-skills. (Prince & Felder, 2006)

3.1.1 EXAMPLES OF USING PHYSICAL LEGO-BRICKS IN EDUCATIONAL ACTIVITIES

3.1.1.1 LEGO for product design and manufacturing simulations

(Grogan & De Weck, 2019) present a hands-on classroom activity using LEGO bricks to simulate product design and manufacturing processes. The activity engages students in a three-tier logistics system involving suppliers, manufacturers, and customers, with real-time data collection facilitated by a simple Product Lifecycle Management system built using Google Forms and Sheets. Key learning objectives include understanding tradeoffs between speed and quality, standardization versus product variety, and the impact of design-for-manufacturing principles. The simulations have been successfully applied in pre-college to professional education settings, demonstrating how LEGO-based activities

can effectively teach abstract engineering concepts while fostering creativity, collaboration, and problem-solving skills.

3.1.1.2 LEGO in software engineering education

Another good example is LEGO® Serious Play. Currently, it is starting to matter to the higher education sector, and it is being applied in several knowledge fields as diverse as marketing, arts, or education. (Lopez-Fernandez et al., 2021) In the engineering field, initiatives carried out in civil engineering, industrial engineering or systems engineering can be highlighted. These contributions report a very positive impact on the student learning process and point out the great applicability of LEGO® Serious Play (LSP) in higher education contexts. (Lopez-Fernandez et al., 2021)

(Lopez-Fernandez et al., 2021) explores the use of LSP as an active learning methodology to teach software engineering concepts. The study involved 242 computer science students who participated in an LSP activity designed to simulate software development life cycles, including waterfall, evolutionary, and incremental models. The activity required students to build LEGO chairs while following specific software engineering processes, such as requirements definition, design, implementation, and validation.

Results showed that students found the activity highly engaging, motivating, and effective for learning software engineering principles and developing soft skills like teamwork and communication (see Figure 13). The study also demonstrated that LSP is as effective as traditional teaching methods in knowledge acquisition while offering additional benefits in student engagement and practical application.

Item	M	SD
The LSP activity enhanced my learning process	4.52	0.60
The LSP activity was motivating	4.47	0.73
The LSP activity was fun	4.73	0.56
The LSP activity was useful to develop my soft skills (communication, teamwork, etc.)	4.56	0.60
The LSP activity was useful to learn SE concepts	4.29	0.76
The LSP activity was useful to learn the key activities in SE (requirements, analysis, design, etc.)	4.47	0.62
The LSP activity was useful to learn the fundamental SE life cycles (waterfall, incremental, evolutionary)	4.55	0.58
The LSP activity contributed to create a positive climate in the classroom	4.74	0.53
I would like to learn through other LSP activities in other courses	4.48	0.80

Figure 13: Learning experience survey results (Lopez-Fernandez et al., 2021)

(Kurkovsky et al., 2019) investigate the use of LEGO-based activities to teach software requirements engineering (RE) in an engaging, hands-on manner. The authors designed two active learning modules: one focused on requirements elicitation and system refactoring in an agile environment, and another on requirements elicitation and analysis using LSP. These modules were tested in undergraduate software engineering courses, with students building LEGO models to represent system features, brainstorm requirements, and simulate stakeholder interactions. Results showed that LEGO-based activities enhanced student engagement, facilitated abstract thinking, and improved understanding of RE concepts. Students reported that the tactile and playful nature of LEGO helped them visualize requirements, collaborate effectively, and think creatively about problem-solving. The study highlights LEGO's potential as a tool to bridge the gap between theoretical RE concepts and practical application in a team-based setting.

(Kuitert et al., 2025) mention LEGO minifigures as an example to teach variability modeling in Software Product Lines. By comparing LEGO's modular components to software features, they demonstrate how different configurations can be assembled from interchangeable parts. The LEGO examples help students to visualize abstract software product line engineering concepts through a familiar, interactive framework.

3.1.2 EXAMPLES OF USING DIGITAL LEGO-BRICKS IN EDUCATIONAL ACTIVITIES

3.1.2.1 Istanbul Technical University, Turkey

(Doma & Şener, 2021) explore the use of LEGO-based CAD tools in online architectural design education, particularly addressing challenges posed by the shift to distance learning during the COVID-19 pandemic. The authors conducted a three-day workshop with thirteen first-year architecture, interior architecture, and industrial design students, who used LeoCAD to design modular living pods for custom LEGO Minifigure personas.

The workshop demonstrated LEGO's effectiveness as a bridge between conceptual design and digital tools in remote education. The authors propose developing a specialized brick-based CAD tool for architectural education, incorporating student feedback to address current software limitations. The students clearly indicated that they LEGO bricks as a core design element have great potential. According the post-workshop survey results (see Figure 14), the majority of them (77%) would like to use digital LEGO in future.

	Mean Response Value (SD)	Response Variables
Design ideation potential with physical LEGO bricks*	4.23 (0.83)	1: Very poor 2: Poor 3: Acceptable 4: Good 5: Very Good
Design iteration potential with physical LEGO bricks†	4.46 (0.78)	
Design ideation potential with LEGO bricks in CAD*	4.38 (0.65)	
Design iteration potential with LEGO bricks in CAD†	4.38 (0.77)	
Advantage of physical LEGO bricks over CAD bricks	3.77 (0.93)	1: Much worse 2: Somewhat worse 3: Stayed the same 4: Somewhat better 5: Much better
Do you prefer LEGO based CAD as a design tool	0.77 (0.44) 3 No, 10 Yes	No: 0, Yes: 1

* $p > .05$, † $p > .05$

Figure 14: Post-workshop evaluation survey results (Doma & Şener, 2021)

3.1.2.2 KinderUni OÖ

KinderUni OÖ (Kinder University Upper Austria) is an educational initiative by IFAU – Institut für Angewandte Umweltbildung that aims to introduce children and young people aged 5 to 15 to science and knowledge through various extracurricular activities. It offers a range of courses, workshops, and excursions, often involving cooperation with universities, universities of applied sciences, and companies. The project was initially

established in Steyr in 2004 and has since expanded to include locations like Almtal, Ennstal, Hagenberg, Linz, and Wels.

The following projects we carried out for competition at the KinderUni 2024 (<https://www.kinderuni-ooe.at/>). The main goal of the competition was to teach kids between 12 and 14 years how to use digital LEGO (Leocad) in combination with CADdrive. The project created by the kids during this competition are illustrated in Figure 15 and Figure 16.

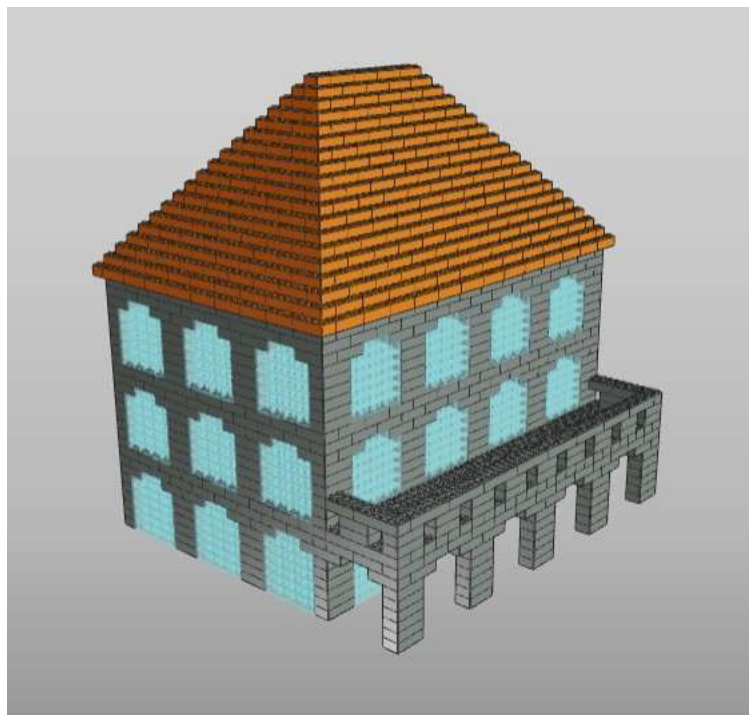


Figure 15: (Hackenberg & Zehetner, 2025)



Figure 16: (Hackenberg & Zehetner, 2025)

3.1.3 COMPARISON OF DIGITAL AND PHYSICAL LEGO

While physical LEGO remains superior for hands-on learning, creativity, and real-world problem-solving, digital LEGO excels in cost, scalability, and advanced design applications, making it ideal for education, prototyping, and remote collaboration, as it is shown in Table 1. However, the best approach may involve a hybrid model—using digital tools for planning and physical LEGO for execution (DeepSeek, 2025).

Table 1 Comparison of digital and physical LEGO (DeepSeek, 2025)

Feature	Digital LEGO (LeoCAD example)	Physical LEGO
Cost	Free/low-cost software No need to buy bricks	Expensive sets Requires continuous investment
Accessibility	Works anywhere with a computer No storage needed.	Requires physical space and storage for bricks

Design Flexibility	Unlimited bricks, Undo/redo, virtual modifications.	Limited by real-world brick availability and physics.
Collaboration	Easy file sharing; Remote teamwork possible.	Requires in-person collaboration.
Portability	Projects saved digitally; Easy to transport files.	Bulky builds; Difficult to move.
Learning Applications	Teaches CAD, 3D modeling, and prototyping.	Enhances tactile skills, creativity, and teamwork.
Prototyping	Easy to iterate and export for 3D printing.	Hands-on testing of physical builds.
Engagement	Better for technical learners and digital workflows.	More engaging for hands-on, interactive learning.
Limitations	No tactile feedback; Requires CAD familiarity.	Limited by brick availability; storage and cleanup needed.

4 FUNDAMENTAL APPROACH

4.1 CASE STUDY DESCRIPTION

The case study is prepared for demonstrating the applicability of the SPLE techniques for the development of physical products in the open-source educational environment CADdrive and the key challenges in applying the principles to such products. The case study draws inspiration from established SPLE practices in drone software. (Steiner et al., 2013) present a catalog of patterns to represent variable features in Simulink and show how to construct a simple software product line for the small-sized drone called Tiriba using two tools to manage variabilities: Pure::variants and Hephaestus. This research adapts variability modeling techniques to a physical drone product line: unlike Simulink-based approaches, digital LEGO is used to visualize constraints, enabling student experimentation, rapid prototyping and immediate feedback.

The key design decision was to focus on First Person View (FPV) drones for their modular design and clear performance trade-offs, breaking them down into key components that were adapted into LDraw-compatible modules. The study's validity metrics include three FPV drone variants (ultralight, freestyle, and long-range) with distinct performance characteristics, thirteen modular components, two types of spatial constraints, two types of interfaces, a feature model and a complete 150% model representation.

The case study was developed using a design science methodology, following three iterative phases:

- Problem identification: Addressed gaps in teaching PLE for physical systems (e.g., lack of spatial constraint modeling tools).
- Artifact design: Built a modular drone LEGO product line encoding interfaces and constraints.
- Validation: Evaluated through SPLE experts' feedback of its practical feasibility.

First, an overview of the atomic modules of the product line from which the individual variants can be assembled is provided. Then, the interfaces between the individual modules and possible interface constraints are explained. Thereafter, the configuration options of the drone use case in the form of a feature model are overviewed. And finally, the 150% digital LEGO model for the drone use case including all atomic modules as well as the assembly structures of the three different drone variants is introduced.

4.2 ATOMIC MODULES

The atomic modules of our drone product line case include a controller, propellers, batteries and frames with covers and cables. Each module adds required system functionality and comes in different sizes (except the controller) with different physical and performance characteristics. In the following, the research explains each class of atomic module as well as their functionality in the context of the drone use case and the available sizes in more detail.

Controller. The controller is shown in Figure 17 and in Figure 18. It serves as the central processing unit of the drone, managing flight dynamics, sensor inputs, and mission execution. Unlike other modules, the controller does not vary in size or hardware across drone variants; instead, its functionality is customized through software configurations tailored to each use case.

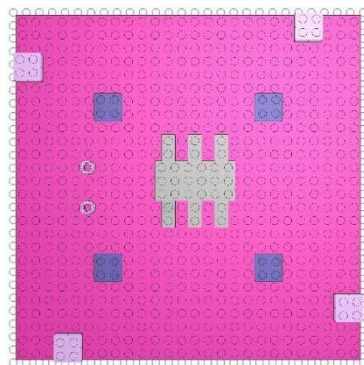


Figure 17: Controller top view

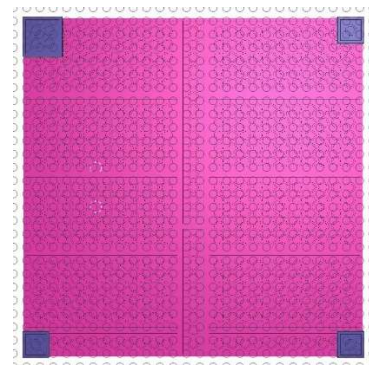


Figure 18: Controller bottom view

Propellers. The propellers are shown in Figure 19, Figure 20 and Figure 21. In the case study, propellers implicitly include motors. They generate thrust by spinning rapidly, allowing a drone to move in all directions. The small and large two-blade propellers provide better efficiency and longer flight times, while the medium triblade propeller offers more stability and maneuverability. In general, smaller propellers provide better agility, while bigger ones provide more thrust and efficiency, allowing the drone fly longer distances. On the contrary, larger propellers also require more power to spin.

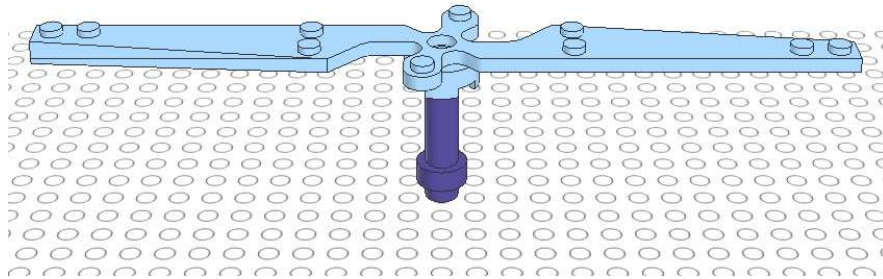


Figure 19: Small two-blade propeller

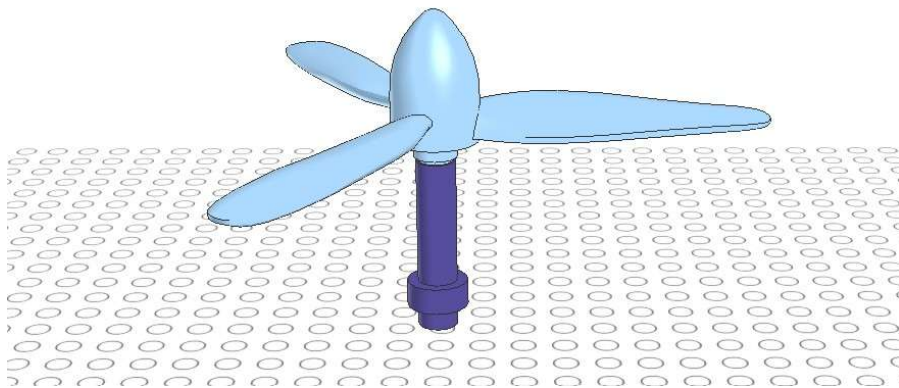


Figure 20: Triblade propeller

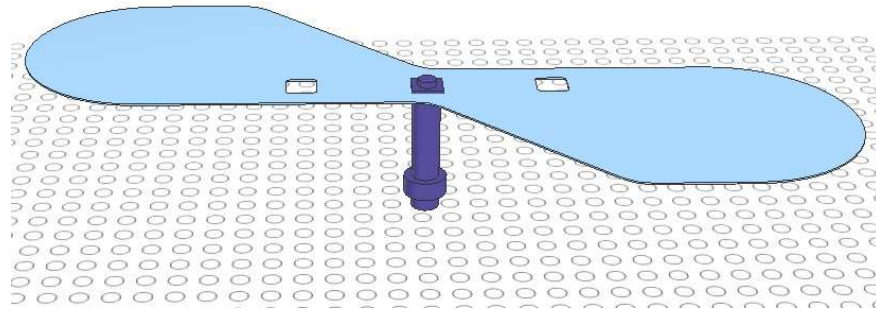


Figure 21: Large two-blade propeller

Batteries. The batteries that are shown in Figure 22, Figure 23, and Figure 24 provide power for the drone and its equipment to operate. The smaller capacity batteries are lighter and more compact, which can improve the agility and maneuverability of the drone and typically comes with lower material, production, and shipping costs. However, smaller batteries typically also have shorter flight times, which means that for longer flights a higher capacity battery is required, which potentially can power additional equipment.

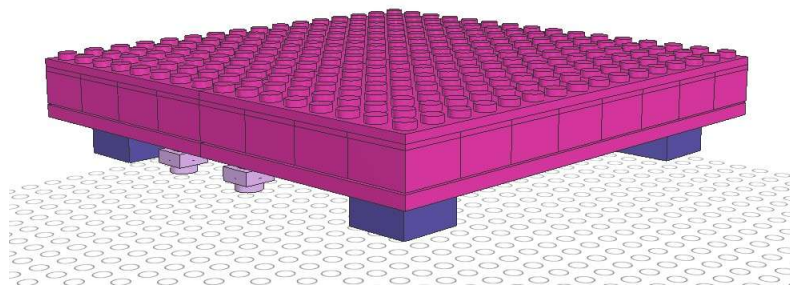


Figure 22: Small battery

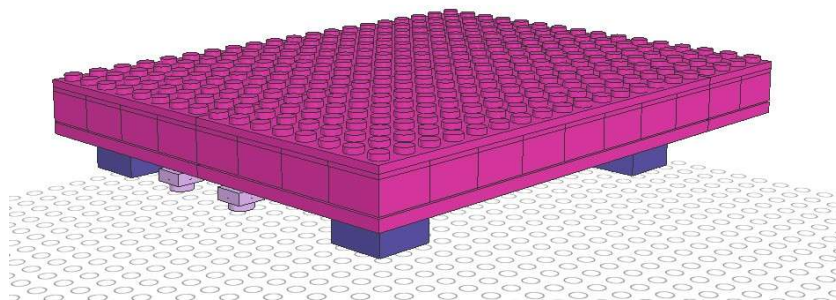


Figure 23: Medium battery

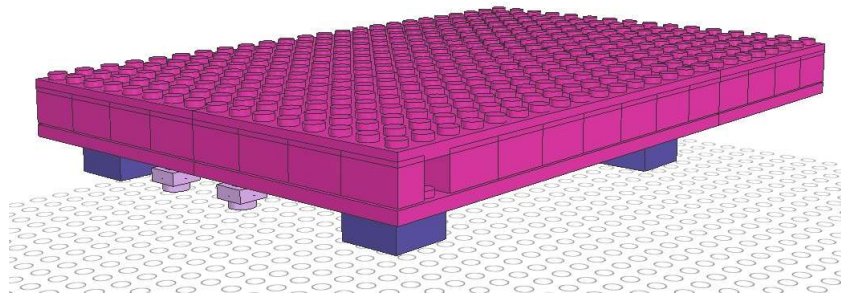


Figure 24: Large battery

Frames, covers and cables. The frames (see Figure 25 and Figure 26), which in the case study implicitly include the legs, serve as the mechanical structures that hold the previous components together. The small frame is more maneuverable and agile due to its lower weight and air resistance. In contrast, the large frame offers more space for additional components such as the larger batteries and possibly other equipment typically mounted on drones (e.g. cameras and infrared sensors).

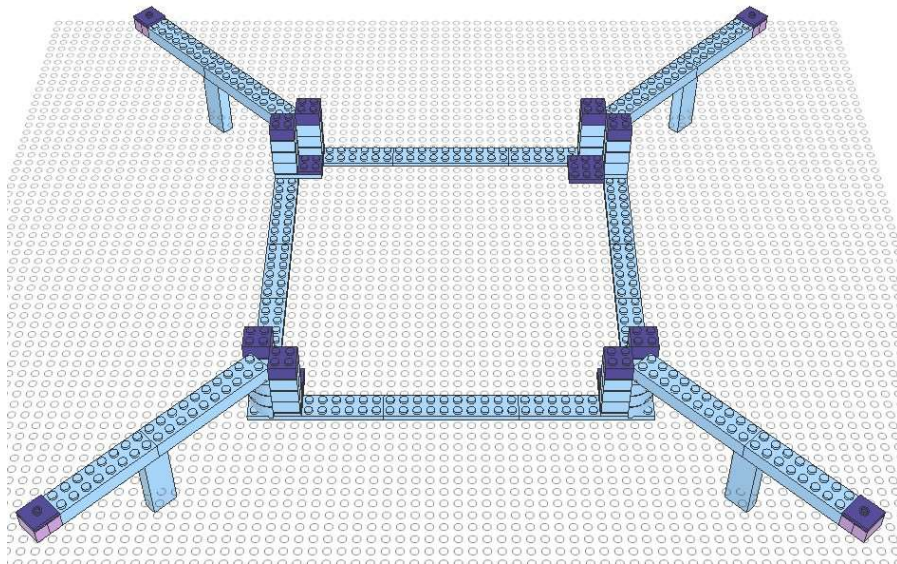


Figure 25: Small frame

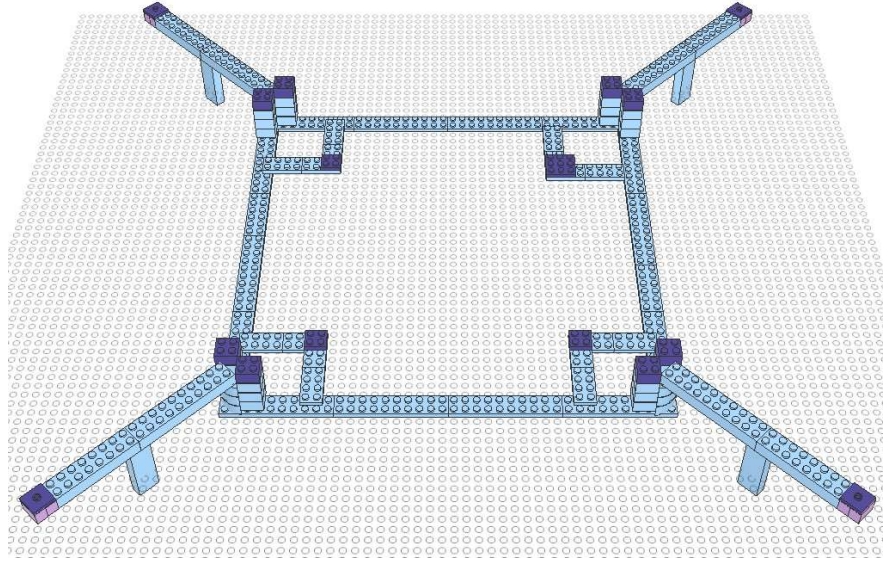


Figure 26: Large frame

The covers (see Figure 27 and Figure 28) mainly protect the internal components of the drone (i.e. batteries and possibly other equipment) from external influences such as humidity. Furthermore, the covers improve the aerodynamic efficiency of the drone due to smoothening the drone surface. In alignment with the previous two frame sizes, the covers come in a small and a large size as well. The small size fits onto the small frame, while the large size fits onto the large frame and, hence, supports larger equipment.

The cables are also shown in Figure 27 and Figure 28. With lengths matching the frame size (small or large), they connect the controller to the motors mounted on the drone's legs, ensuring proper power delivery and signal transmission.

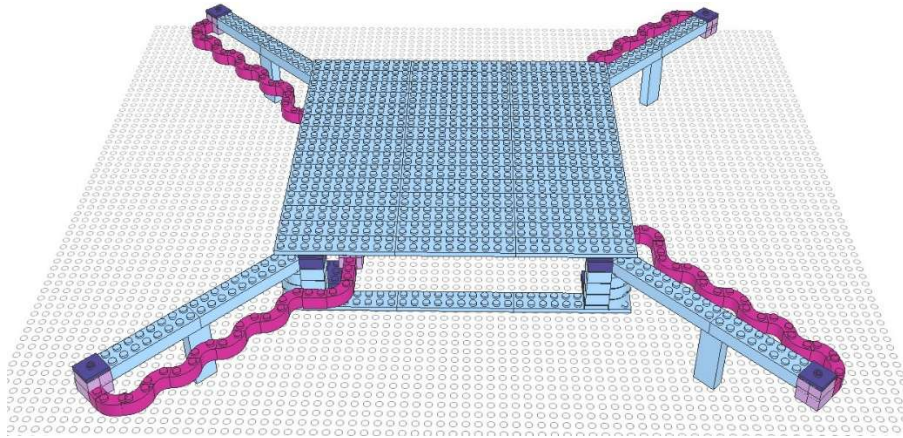


Figure 27: Small frame with the small cover and shorter cables

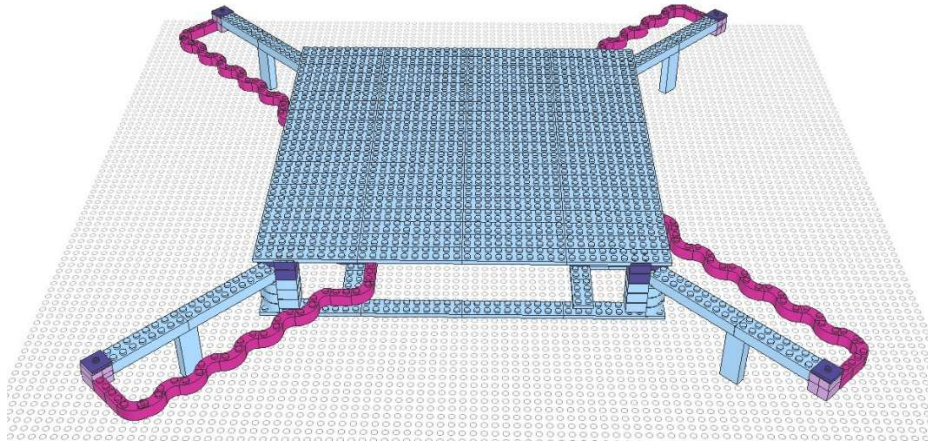


Figure 28: Large frame with the large cover and longer cables

4.3 MODULE INTERFACES

The interfaces maintain the drone's modular design philosophy while ensuring reliable mechanical integration and electrical functionality across all size variants. As stated before, the frame modules are responsible for holding the other modules together. Consequently, the frame modules interface with all the other modules except battery modules. In the following, we describe the mechanical and electromechanical interfaces between the modules.

Mechanical interfaces

All the mechanical interfaces are represented by dark-blue studs and shown in Figure 29.

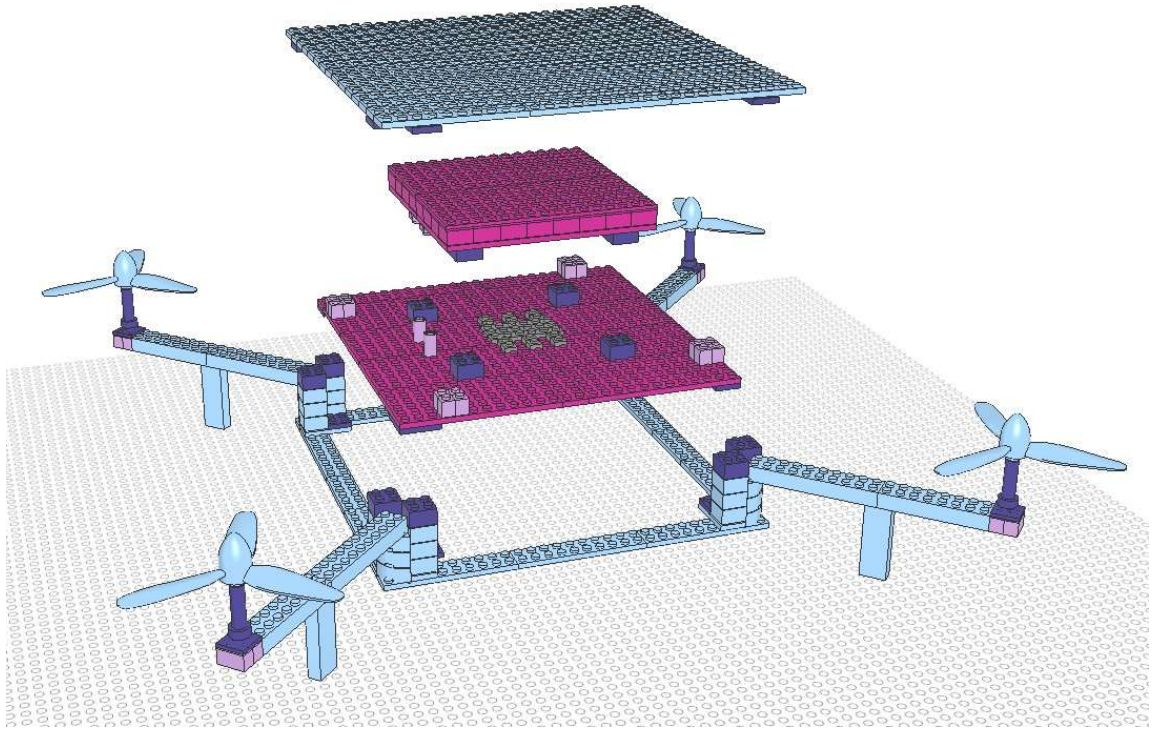


Figure 29: Mechanical interfaces of a drone are represented by dark-blue studs

Frame-cover interface. The small and the large frames also provide an interface for the small and the large cover. Here, the small cover only fits onto the small frame, which can be verified through checking the stud connections after mounting. In contrast, the small cover does not fit onto the large frame, because certain studs remain unconnected after mounting. Similarly, the large cover only fits onto the large frame, while not fitting the connector interface of the small frame as required.

Frame-controller interface. It serves as the mechanical connection point that securely mounts the controller module to the drone frame. Both the small and large frame variants

incorporate standardized mounting features designed to match the controller's base dimensions.

Frame-propeller (motor) interface. Both, the small and the large frame, provide standardized mechanical connectors for the propellers. In principle, due to the standard connector geometry each propeller size (i.e. small, medium, and large) can be mounted onto each frame size (i.e. small and large). However, when mounting large propellers onto small frames, the propellers collide with these frames in certain rotational angles and, hence, yield undesirable system states.

Electrical and electromechanical interfaces

Electrical and electromechanical interfaces are represented by lilac studs and shown in Figure 30.

Controller-battery interface. The controller provides a standard interface for batteries represented by a standard seat in the digital LEGO models. It includes four dark-blue studs, which enable mechanical stability, and two round lilac pins, which provide electrical connection. Furthermore, all battery sizes (i.e. small, medium, and large) are compatible with this standard seat.

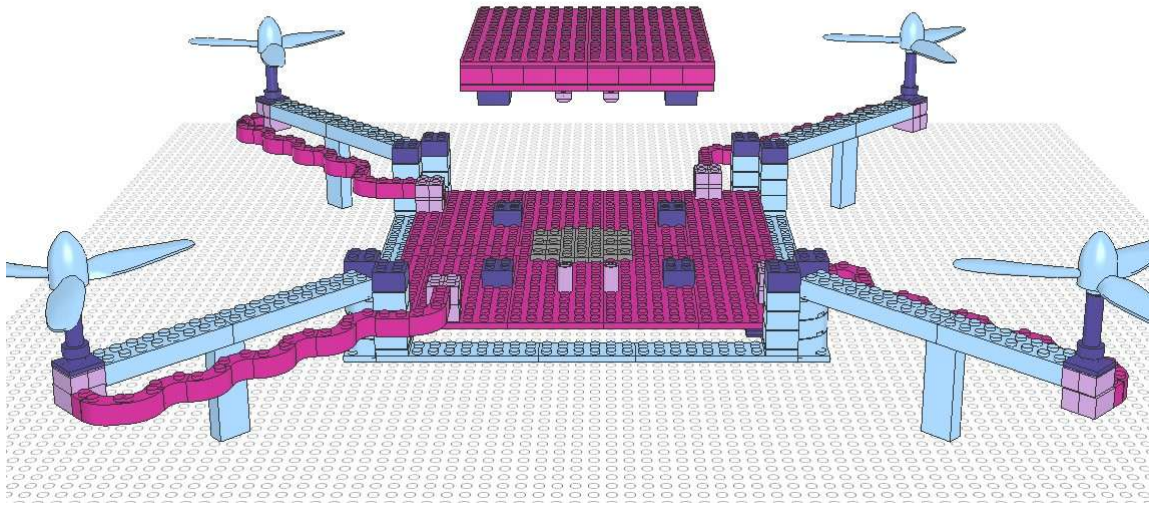


Figure 30: The electromechanical interfaces are represented by lilac studs

Controller-propeller interface (wiring). It represents the critical electromechanical connection that delivers both power and control signals from the controller to the motors driving the propellers. This interface utilizes standardized electrical connectors on the controller's output ports, which mate with corresponding connectors on the motor wiring. The cable lengths are precisely sized to match the frame dimensions - shorter for small frames and longer for large frames - to maintain proper tension without excess slack. All propeller sizes share identical electrical connection specifications, while the controller's software automatically adjusts operational parameters.

4.4 VARIABILITY MODELING

As stated before, the atomic modules can be assembled according to the interfaces to build customized end products with various different performance characteristics. However, not all combinations of atomic modules are feasible due to geometric and functional constraints. The set of feasible configurations for the drone case study is described in the form of a variability model in Figure 31. Feature models were chosen to describe configurations due to the following reasons:

mandatory grouped features that must all be included together, and "Mandatory" identifies required components.

Furthermore, the model also references "require constraint dependency" and "exclude constraint dependency" to capture additional configuration rules beyond the basic hierarchy, though these constraints are not visually detailed in the image. This feature model organizes the drone's variability into a clear parent-child structure that makes configuration dependencies immediately apparent. The constraint dependencies state that the small frame together with the large propeller and/or the large battery do not form a feasible product configuration.

Note that in the drone case the constraint dependencies encode the (in-)compatibilities of atomic modules due to their spatial properties. In the following, static and dynamic space constraints are distinguished.

Static space constraints

Static space constraints concern the static (i.e. invariant) spatial properties of modules. In the drone case, this type of constraint applies to the frames and the batteries, where only the large battery is compatible with the large frame and only the small and the medium batteries are compatible with the small frame due to space limitations.

Dynamic space constraints

Dynamic space constraints concern the dynamic (i.e. variable) spatial properties of modules. In the drone case, this type of constraint applies to the small frame and the large propellers. In general, also the large propellers can be mounted into small frames due to standard connector geometries. Also, in certain rotational angles, the large propellers do not collide with the small frame. However, during drone operation other rotational angles are possible, which cause such collisions, impair system functionality, and potentially cause damage.

4.5 150% DRONE MODEL

In order to represent all the possible combinations of features for all the possible drone variants, a 150% model is created out of digital LEGO. As shown in Figure 32, the model contains more information than what is needed for a specific product, for instance, it includes all the three propeller options, even though any finished drone will only have one specific propeller type. The model serves as a comprehensive master blueprint that encompasses the complete range of design possibilities for a drone product line. This model systematically incorporates every potential feature and variant, establishing a complete reference framework for the entire product line.

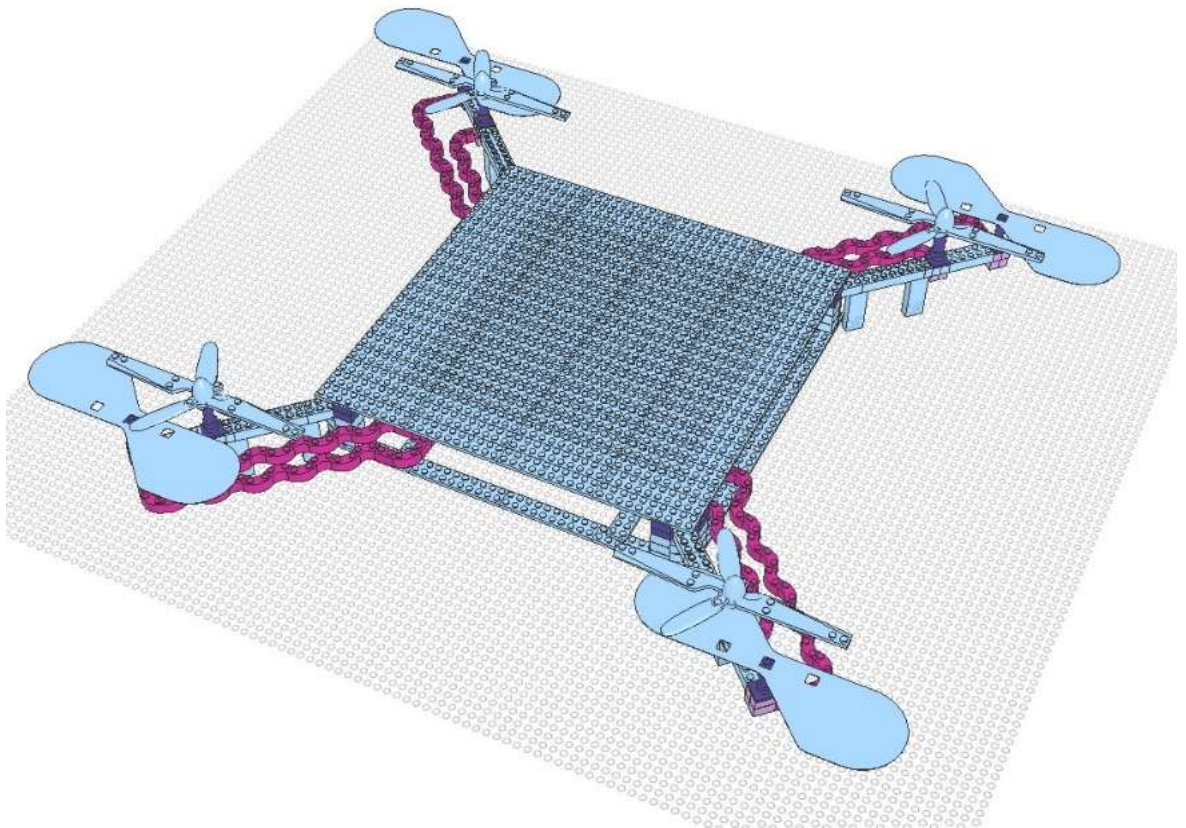


Figure 32: 150% model for the drone product line

At its core, the model captures all commonalities - those fundamental components that every drone configuration must include, such as the controller module. This universal element forms the foundation upon which all product variants are built. Moreover, the

model exhaustively documents all possible variabilities across the product line. It includes the full range of physical component options, from propeller types with different blade configurations and size variations to performance specifications such as different battery capacities.

The 150% model is created in LeoCAD, which adheres to the LDraw standard. The LDraw file contains multiple assemblies. Each assembly contains multiple references, which point to either a model (which itself can contain sub-assemblies) or bricks (which are standardized and belong to a brick catalog). The reference includes transformation data (position, orientation) and optional appearance data (color). Within the structure of the LDraw file, a main model serves as the foundation where all the possible variants of the drone are instantiated from. Each variant consists of modular components. These modules are organized within the same file. The approach enables the reuse of the modules, promoting efficiency and versatility in design.

4.6 MAPPING THE VARIABILITY MODEL WITH THE 150% MODEL

The 150% drone model goes beyond simply listing components by also capturing the complete set of configuration rules and constraints that ensure valid product combinations. These rules guarantee that any derived product variant will be technically feasible and operationally sound.

To ensure this, the research proposes to create a configuration space within an LDraw file. The idea is, to utilize the LDraw structuring approach, giving specific names to every submodel of the LDraw hierarchy. The submodel here plays a feature container role, representing a node in the 150% model. Its purpose is to encode features from the feature model. These submodels must be instantiated only once in the main model, follow a specific naming convention that encodes feature information, form the hierarchical structure of the feature tree, and may or may not have physical 3D content, distinguishing between abstract and concrete features. The names encode the configuration rules. The research proposes a naming scheme that reflects the entire information from the feature model. In addition to a traditional unit name, each name contains in brackets a combination of codes. The codes are described in Table 2.

Table 2 Feature codes, their symbolic representations and meanings

Code abbreviation	Symbolic representation	Meaning
M	!!	Mandatory: child feature must be selected
O	?	Optional: child feature can be selected or not
AF	A	Abstract feature: has no 3D representation
CF	©	Concrete feature: has a 3D representation
OR	No representation	At least one feature must be selected
XOR	No representation	Only one feature must be selected
AND	No representation	All the child features must be selected
EXCL	← - - →	Excludes constraint dependency
REQ	- - →	Requires constraint dependency

The LDraw structure represents a drone system with preset configurations and customizable features. The root is "FPV Drone," which contains two main branches: "Features" and "Configurations".

The "Features" section defines customizable components with constraints. The Controller is a mandatory concrete feature (M_CF), meaning it must be included with no alternatives. The Propeller Type is a mandatory abstract feature with an XOR-group (M_AF_XOR), requiring at least one selection from its sub-features: either Two-Blade (an optional abstract feature, AF_XOR) with concrete options for Small or Large Two-Blade Propellers (both CF), or a standalone Three-Blade Propeller (CF). The Large Two-Blade Propeller has an exclusion constraint (EXCL) with Small Frame&Cover&Cables.

Battery Size is another mandatory abstract XOR-group (M_AF_XOR), forcing a choice between Small, Medium, or Large Battery (all CF). The Frame&Cover&Cables Size is also a mandatory XOR-group (M_AF_XOR) with two concrete options: Small (CF), which excludes both Large Two-Blade Propeller and Large Battery, and Large (CF), which requires (REQ) the Large Two-Blade Propeller.

Key constraints include: the Large Two-Blade Propeller cannot pair with a Small Frame, the Large Frame demands the Large Two-Blade Propeller, and the Small Frame is incompatible with a Large Battery.

Under "Configurations," there are three predefined drone types: Ultralight Drone, Freestyle Drone, and Long-Range Drone. Each configuration name also contains a mathematical formula, which is usually a sum of concrete features.

While each concrete feature in the configuration space is represented by a 3D model in 150% model, abstract features are submodels which have no 3D representation. Only the names of such submodels have mathematical meaning. More submodels could be created to store additional information in their names, for example, a complex formula that describes another configurations or additional constraints.

- FPV Drone
 - Configurations
 - Ultralight Drone ((CF) Small Frame&Cover&Cables + (CF) Small Battery + (CF) Small Two-Blade Propeller + (M_CF) Controller)
 - Freestyle Drone (...)
 - Long-Range Drone (...)
 - Features
 - (M_CF) Controller
 - (M_AF_XOR) Propeller Type
 - (AF_XOR) Two-Blade
 - (CF) Small Two-Blade Propeller
 - (CF) Large Two-Blade Propeller (EXCL Small Frame&Cover&Cables)
 - (CF) Three-Blade Propeller
 - (M_AF_XOR) Battery Size
 - (CF) Small Battery
 - (CF) Medium Battery
 - (CF) Large Battery
 - (M_AF_XOR) Frame&Cover&Cables Size
 - (CF) Small Frame&Cover&Cables: (EXCL Large Two-Blade Propeller, EXCL Large Battery)
 - (CF) Large Frame&Cover&Cables (REQ Large Two-Blade Propeller)

Thus, the model integrates a traditional feature diagram with a 150% model. Based on a certain naming and structure convention, it eliminates the need for separate files to represent the feature model and the 150% model.

4.7 AN ALGORITHM OF DERIVING A 3D MODEL FOR A SELECTED CONFIGURATION

The input is a configuration space with the following characteristics:

- An LDraw model tree with an integrated feature module and configurations.
- 150% drone 3D model.

The procedure contains the following steps:

- The algorithm scans the hierarchical structure starting from the root node.
- The algorithm processes each node:
 - If the node is mandatory (M), it retains it and recursively processes its children. When a child's name which does not match a feature in the selected configuration is found, the algorithm deletes it.
 - If the node is optional (O), it checks if its name matches any feature in the selected configuration. If matched, it keeps the node and processes its children. If not matched, it deletes the node and its entire subtree, and hides the respective 3D model.
 - The algorithm distinguishes between OR, XOR and AND operators and returns a respective abstract feature.

The output is the filtered LDraw tree, which now represents the 3D model tailored to the chosen configuration.

5 USER INTERFACE PROTOTYPING

This chapter builds upon the case study introduced earlier, focusing on the drone configuration system. During the simulation of the configuration process, it became evident that the existing workflow lacked a dedicated user interface to facilitate user interactions. In order to close the gap, the thesis proposes a UI design that aligns seamlessly with the system's logical structure and operational requirements.

To demonstrate how to derive product variants from the 150% model with the usage of CADdrive, the case study was transformed into an exercise for students. The goal is, to adopt SPLE approach incrementally leveraging existing drone assets. The exercise begins with the construction of an ultralight drone. The assets used for the ultralight drone are then adapted for the construction of a freestyle drone, and subsequently for a long-range drone. The first version of the drone product line emerges once the first two drones have been built. This evolves over time by adding the long-range drone and potentially other new drones.

5.1 ITERATION 1: ULTRALIGHT DRONE

The development process is iterative and begins with analyzing customer needs and deriving technical characteristics for the ultralight drone. To perform it, the milestone “Requirements definition” is created, as illustrated in Figure 33.

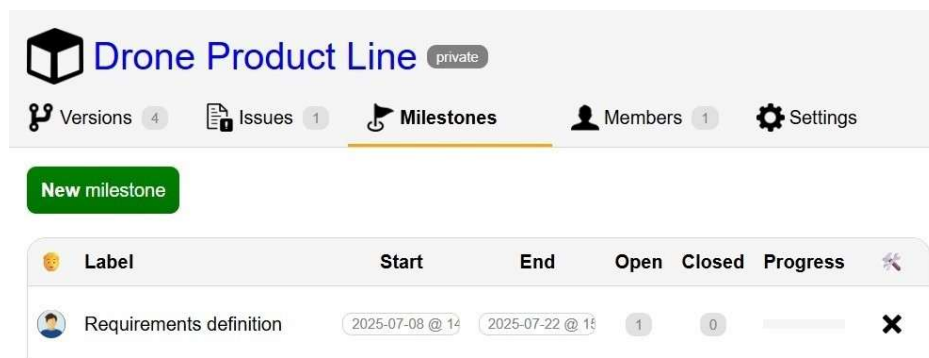


Figure 33: First drone milestone created in CADdrive

As it is shown in Figure 34, it includes only one issue.

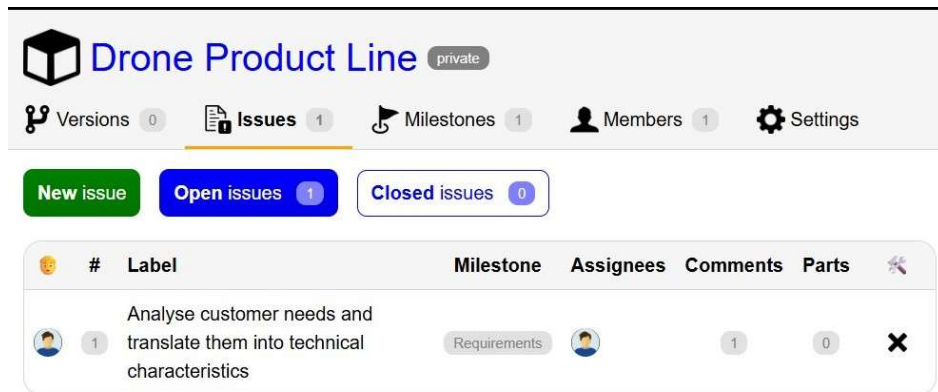


Figure 34: First drone issue created in CADdrive

The outcome is, to formulate key technical characteristics of the drone and associate drone modules with them (see Figure 35).

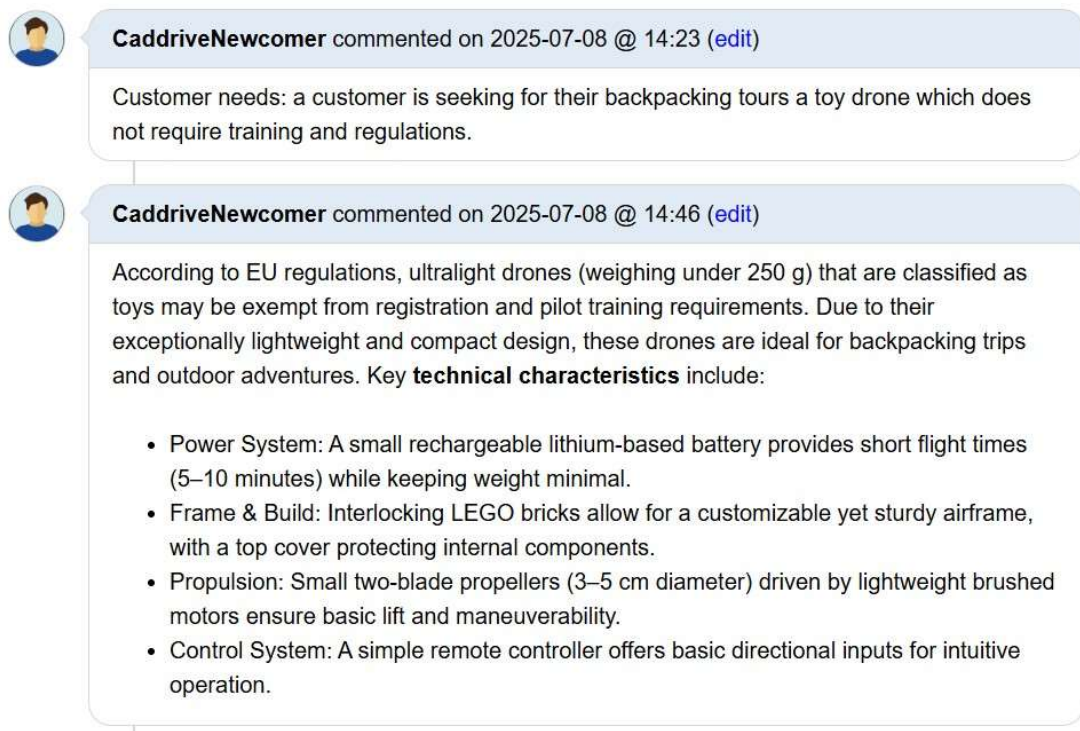


Figure 35: Requirements defined for the ultralight drone

Next, 3D models of modules and assemblies should be built. A milestone “Building a 3D model” is created and it includes four issues, as demonstrated in Figure 36.

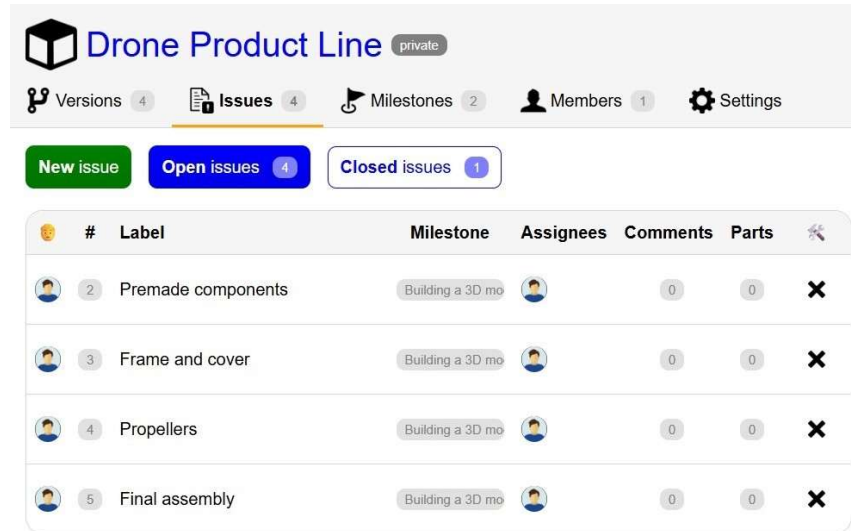


Figure 36: Issues included to “Building a 3D model” milestone

The issues are as follows:

Premade components: The objective is to analyze which modules can not be built out of LEGO bricks (e.g. batteries and the controller) and find available options on the market. 3D models of such components are to be built and uploaded to CADdrive Versions.

Propellers: It is to develop a 3D model which comprises a two-blade propeller and a motor. Both components are available as LEGO parts.

Frame&Cover&Cable: The objective is to design lightweight, compact 3D models of a drone frame, a top cover and cabling that provide sufficient internal space for the controller and battery while minimizing overall dimensions and weight. The design must account for mechanical and electromechanical interfaces between modules, ensuring proper cable routing to connect the controller to the propellers mounted on the frame’s legs. The finalized 3D models appear in CADdrive Versions.

Final assembly: The objective of the issue is to integrate all the modules into the 3D model of the ultralight drone to share it in CADdrive Versions (see Figure 37).

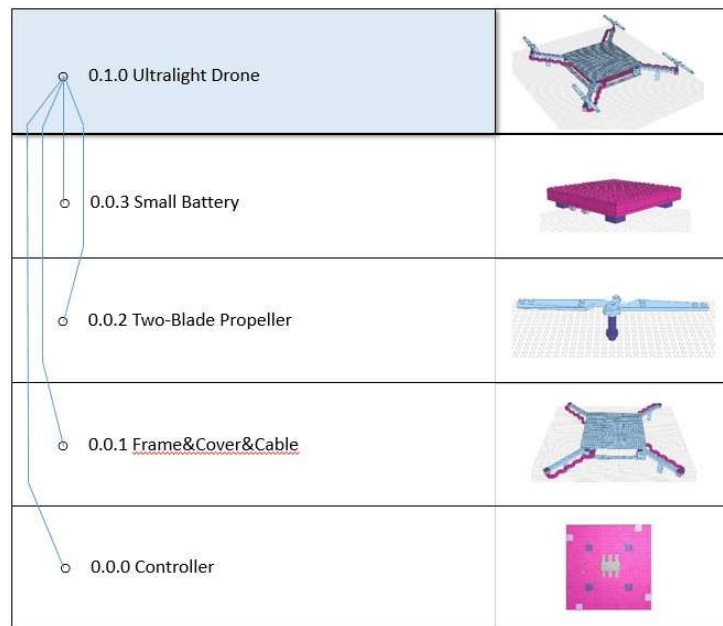


Figure 37: CADdrive's Drone Versions

5.2 ITERATION 2: FREESTYLE DRONE

Again, the development process starts with analyzing customer needs and translating them into technical specifications, as illustrated in Figure 38 and Figure 39.

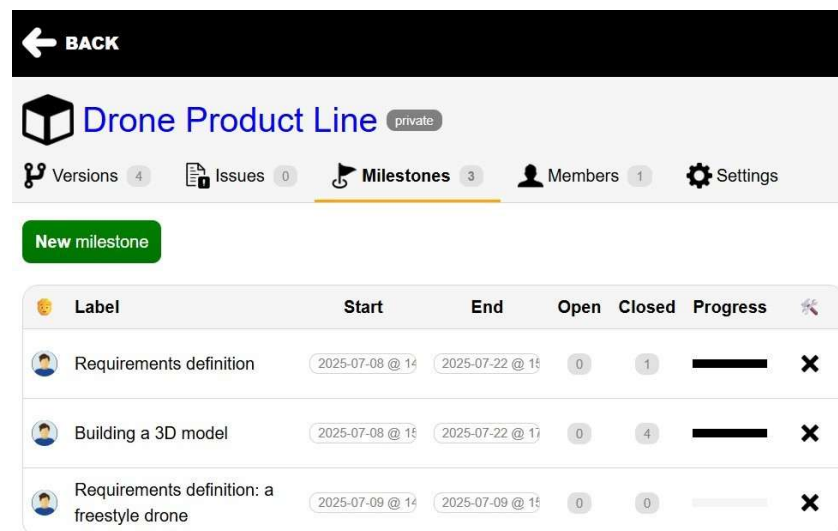


Figure 38: Requirements definition milestone for the freestyle drone

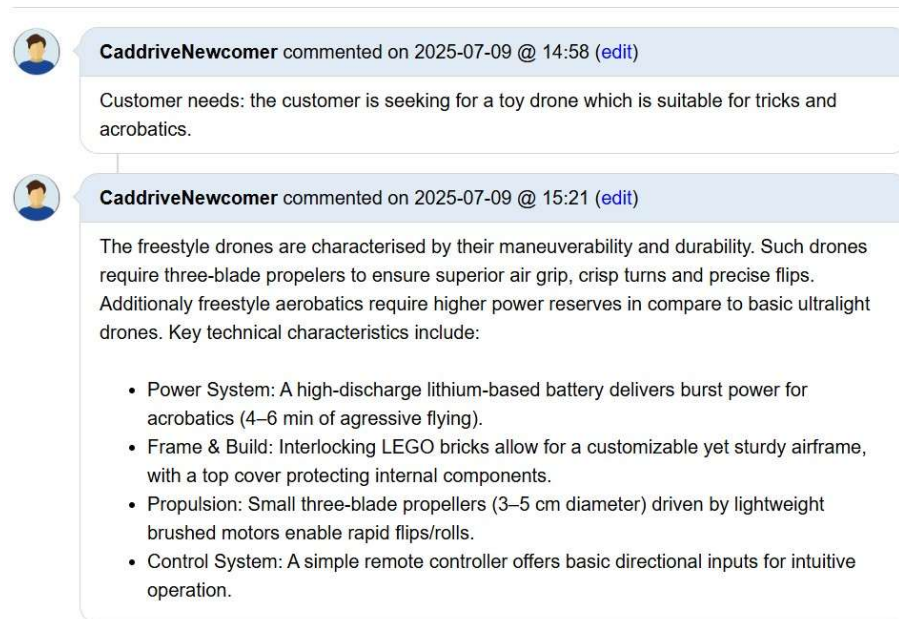


Figure 39: Requirements defined for the freestyle drone

However, unlike the ultralight drone, the freestyle variant must prioritize component reuse from the ultralight drone while incorporating two different elements—specifically, a higher-capacity battery and a triblade propeller for improved maneuverability. This stage necessitates the introduction of a feature model and a 150% model. For that purpose, “Variability modelling” milestone is created (see Figure 40), it contains one issue “Model variability to maximize reuse” (see Figure 41).

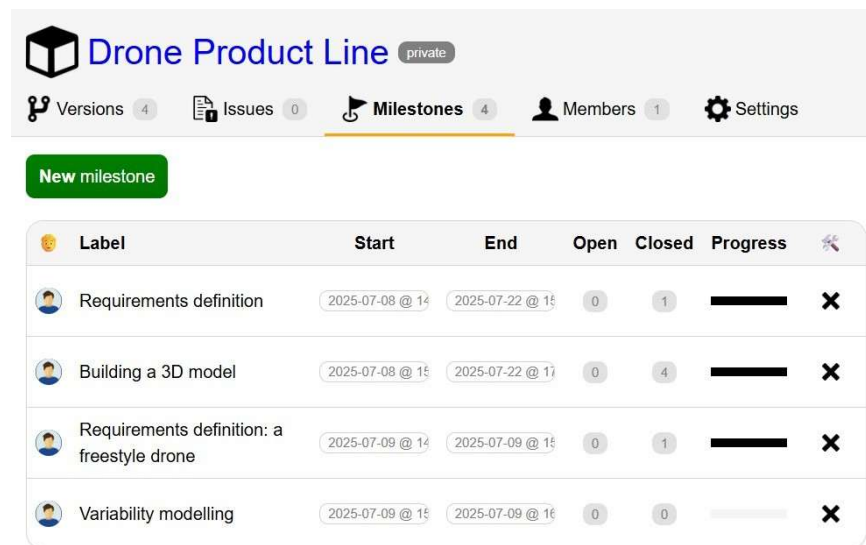


Figure 40: Milestone “Variability modelling”

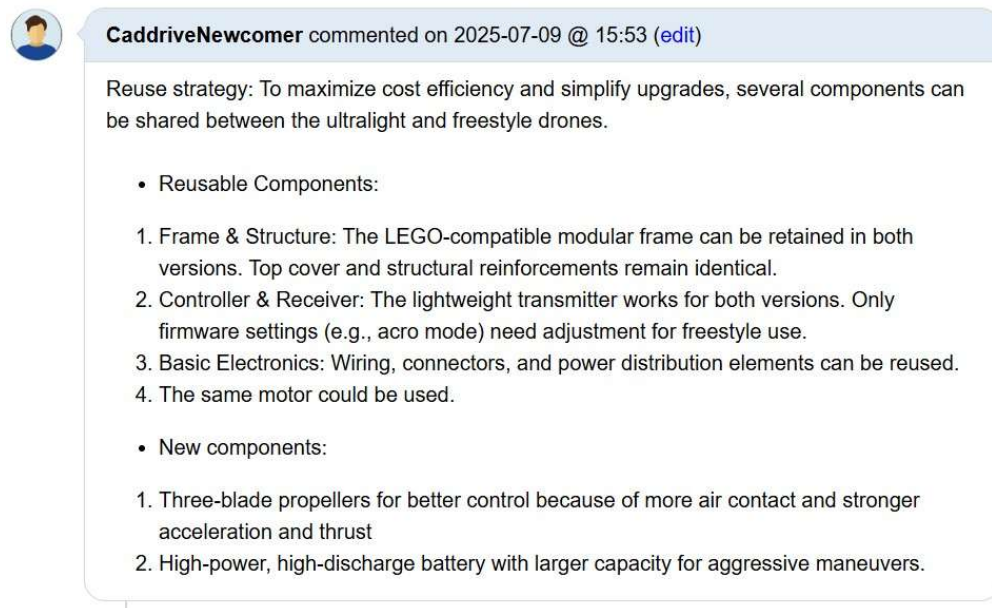


Figure 41: ‘Model variability to maximize reuse’ issue

To achieve the variability, a feature model is developed. It enables to guide the adaptation of shared components and ensures seamless integration of the new elements (see Figure 42). Variability modelling happens in CADdrive’s feature modelling space, described in 5.4.

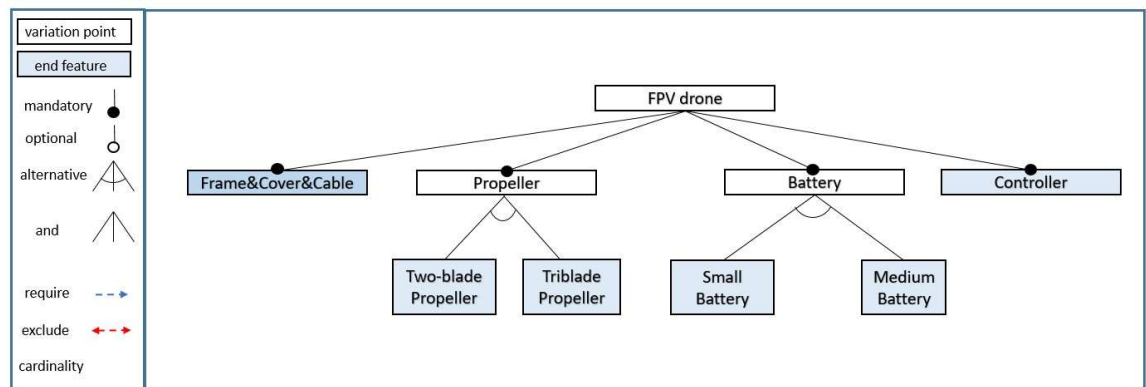


Figure 42: A feature model that comprises features of the ultralight and freestyle drones

The next step is building 3D models for missing components of the freestyle drone, e.g. the triblade propeller and the medium-size battery, and the 150% model (see Figure 43). This happens in CADdrive's configuration space, described in 5.5.

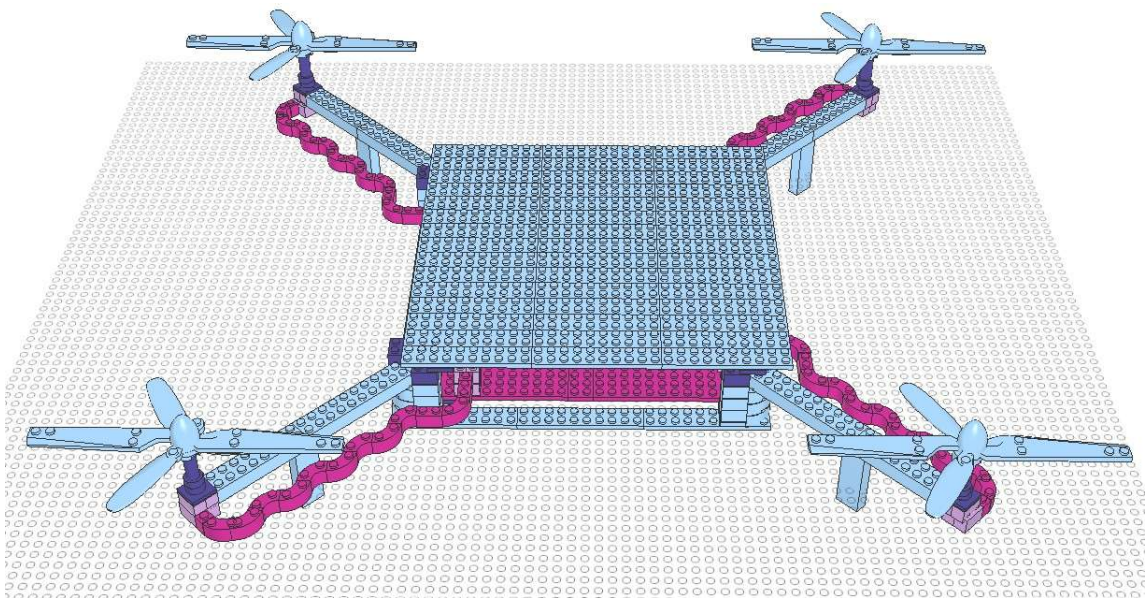


Figure 43: A 150% model that comprises features of the ultralight and freestyle drones

In Versions first version of the Drone product line appears under the number 1.0.0. It comprises two drones: the ultralight drone and the freestyle drone. The updated Version tree is shown in Figure 44.

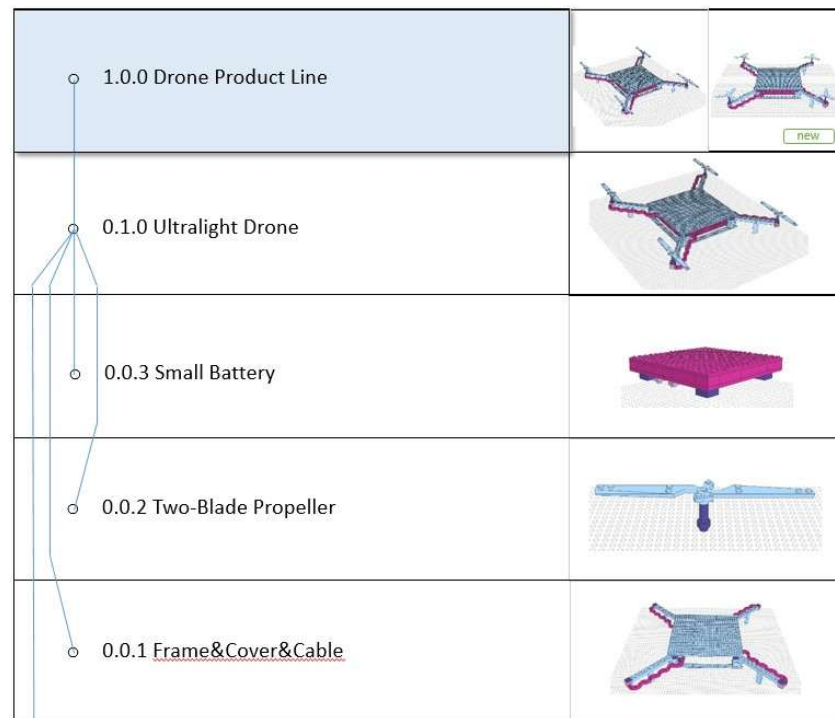


Figure 44: CADdrive's Versions tab with the first version of the Drone Product Line

5.3 ITERATION 3: LONG-RANGE DRONE

The development process for the long-range is similar to that of the freestyle drone. The process starts again with analyzing customer needs and translating them into technical specifications, as it is shown in Figure 45.

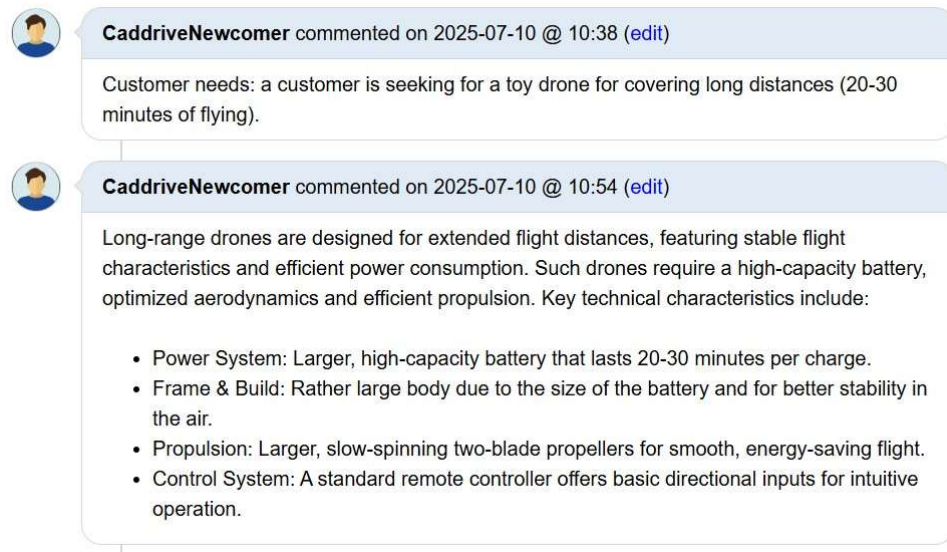


Figure 45: Requirements defined for the long-range drone

As the drone is designed for covering long distances, it reuses only one component from the two previous drones, the controller, while all other 3D models should be built from scratch. At this point, the feature model is further developed and the 150% model is enriched with missing components, e.g. the large two-blade propeller, the large-size battery and the large frame with the large cover and longer cables (see Figure 46 and Figure 47).

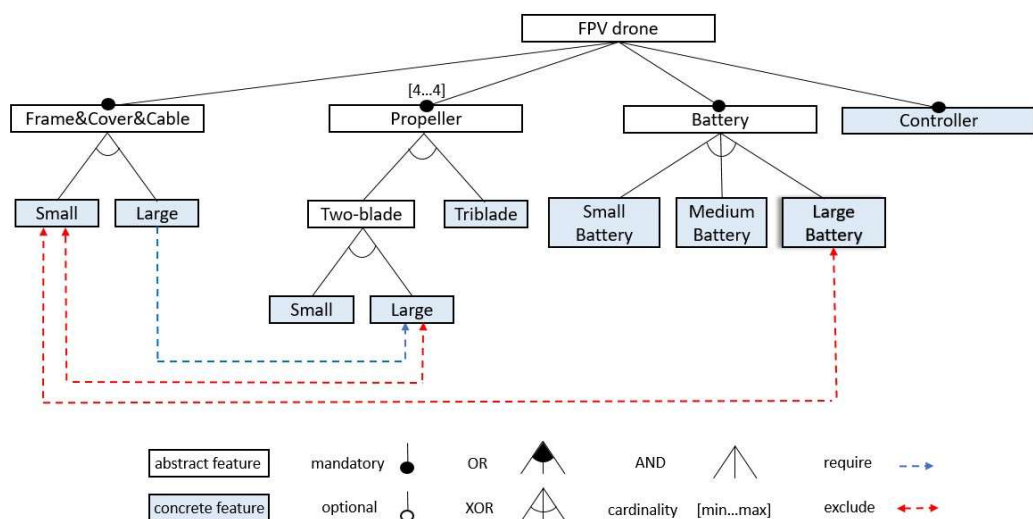


Figure 46: A feature model that comprises features of the ultralight, freestyle and long-range drones

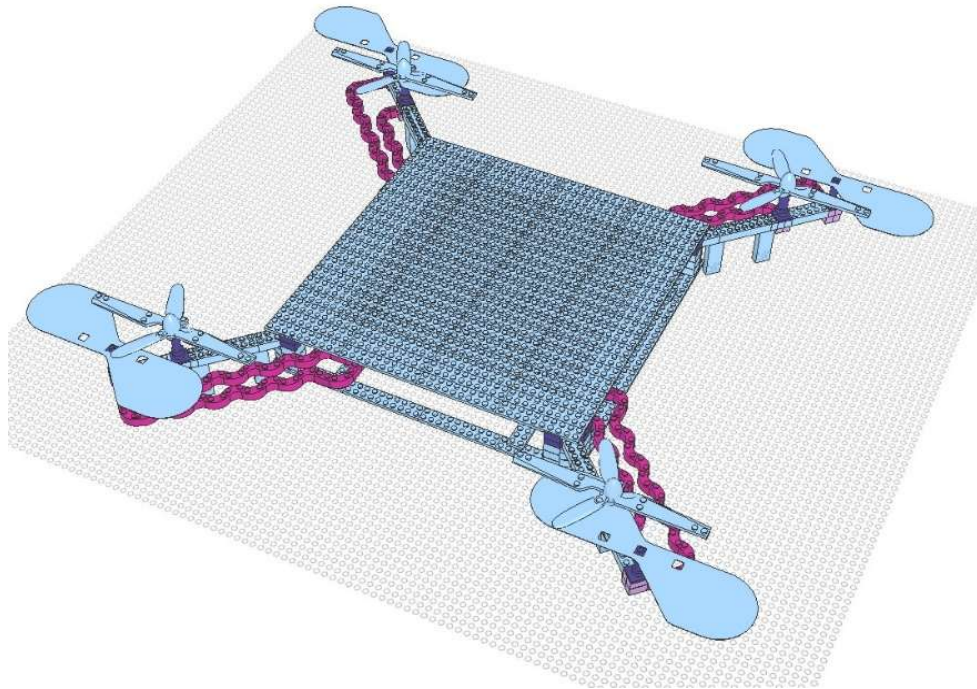


Figure 47: A feature model that comprises features of the ultralight, freestyle and long-range drones

As soon as the components are added to the configuration space and users configure a new variant, another version of the Drone product line appears in CADdrive Versions under the number 2.0.0. It comprises three drones: the ultralight, the freestyle and the long-range drones. The updated Version tree is shown in Figure 48.

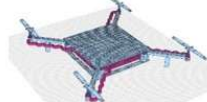
2.0.0 Drone Product Line	 
1.0.0 Drone Product Line	 
0.1.0 Ultralight Drone	
0.0.3 Small Battery	

Figure 48: CADdrive's Versions tab with the second version of the Drone Product Line

5.4 MOCK-UP UI FEATURE MODEL TAB

CADdrive enables users to create feature models within the Feature model tab, as it is shown in Figure 49.



Figure 49: CADdrive IU top navigation bar

The feature modeling interface is used for defining and managing the variability of a product line. The interface is structured into several key sections for variant management.

A panel on the right enables users to place features and then configure a selected feature. It displays the feature's properties through attributes including Concrete/Abstract nature, Mandatory/Optional status, and its relationship type (OR, XOR, AND). The Requires and Excludes fields define dependency constraints. As an example, attributes of the "Large Battery" feature are shown in Figure 50.

The central section is the main working space where feature models are built. Users can add features above and below selected, add cardinality numbers, and delete features.

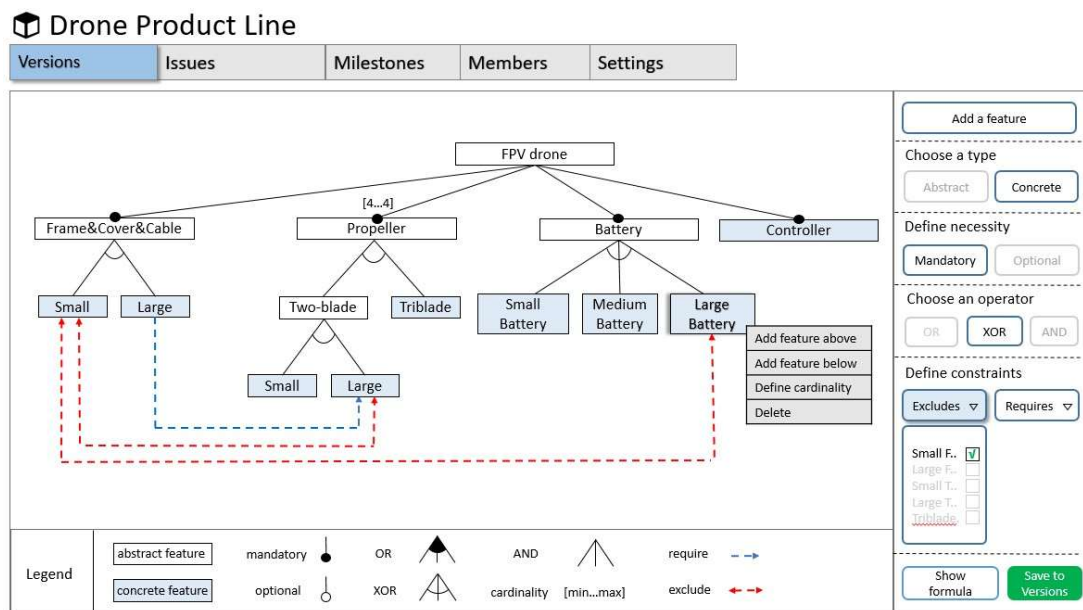


Figure 50: Feature modelling space

A Legend decodes the visual notation used in the model, defining symbols for abstract versus concrete features, mandatory versus optional inclusion, cardinality, and relationship types like alternative, and, require, and exclude.

Clicking the “Show formula” button users can automatically create a formula, which describes a respective variability model according to the naming convention defined in 4.6. It is also possible to enrich the formula with additional information about configurations or constraints by using the Formula editor, as it is shown in Figure 51. This formula is to be uploaded to CADdrive’s configuration space later on. A “Save to Versions” button serves the purpose of storing current changes and data to CADdrive’s Versions.

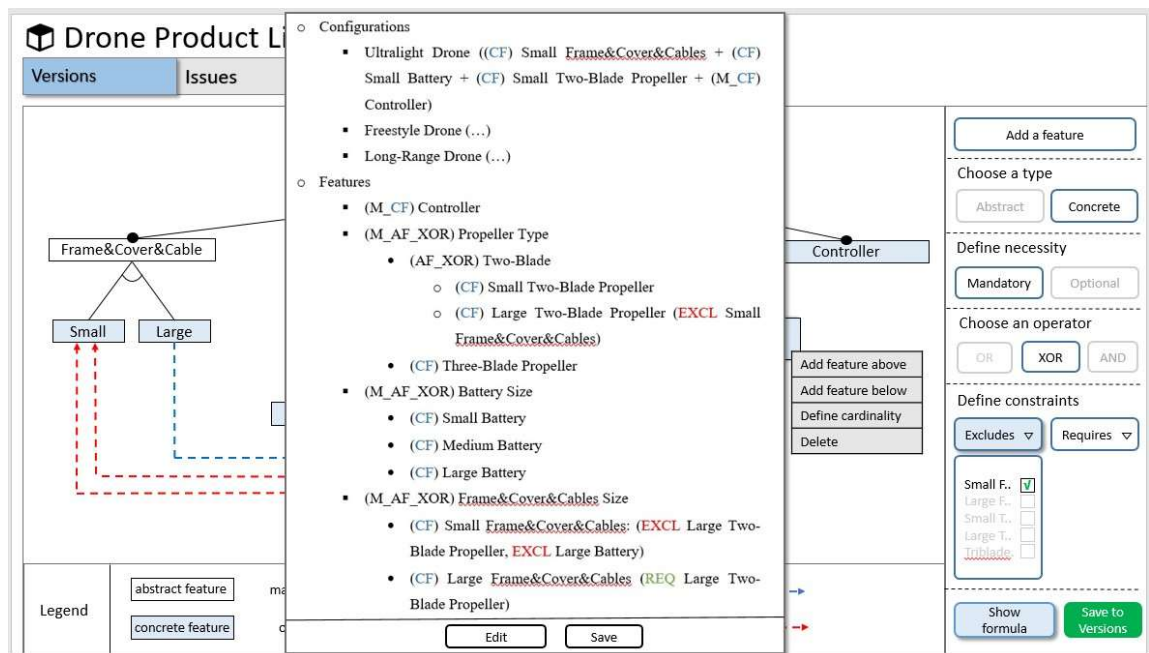


Figure 51: Formula editor

5.5 MOCK-UP UI CONFIGURATION SPACE

The panel on the left is dedicated to browsing, searching, and organizing the components that can be used to build drones. The Parts section header shows "All parts" (modules and bricks, not divided into categories), "In use" (to show only components actively selected

in configurations), “Assemblies” (pre-built modules such as Batteries, Propellers, etc.), “Favorites” (for a user-curated list) and others. A “Search Parts” bar is available for quickly finding components. Under the list there is also a palette of available colors.

The 3D models are displayed in the workspace in the center. Users can add parts to the workspace by selecting components from the list on the left. These parts can then be organized into modules or assemblies as separate LDRAW files, which can be opened independently under different tabs. Figure 52 shows this with the tabs “Battery Small”, “Controller”, and so on.

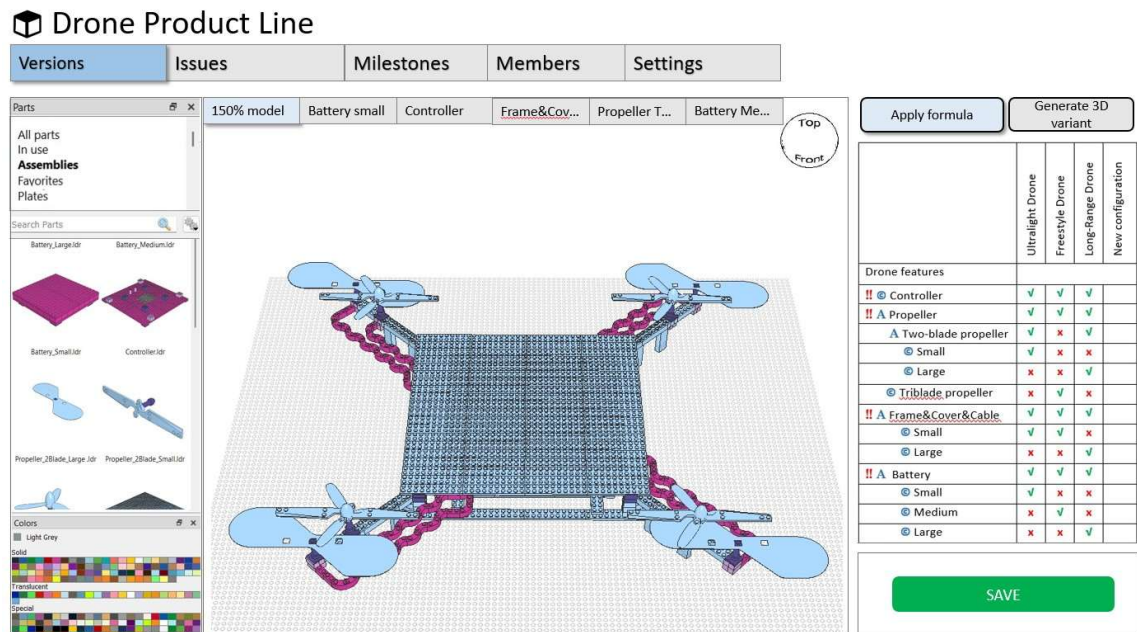


Figure 52: CADdrive’s configuration space

CADdrive enables users to add feature model formulas right in the configuration space using the “Apply formula” button on the top right. Upon applying, users can see a table, which combines the information from the feature model with the information about configurations. Users can also add new configurations selecting relevant components from the table. As the user makes selections, the system dynamically evaluates and updates the available options in real-time. However, the transfer of these changes to the initial formula

is possible only manually. The "Generate 3D variant" button creates or updates a 3D model in a connected CAD system based on the current part selections and a relevant for a specific variant feature model (see Figure 53 and Figure 54).

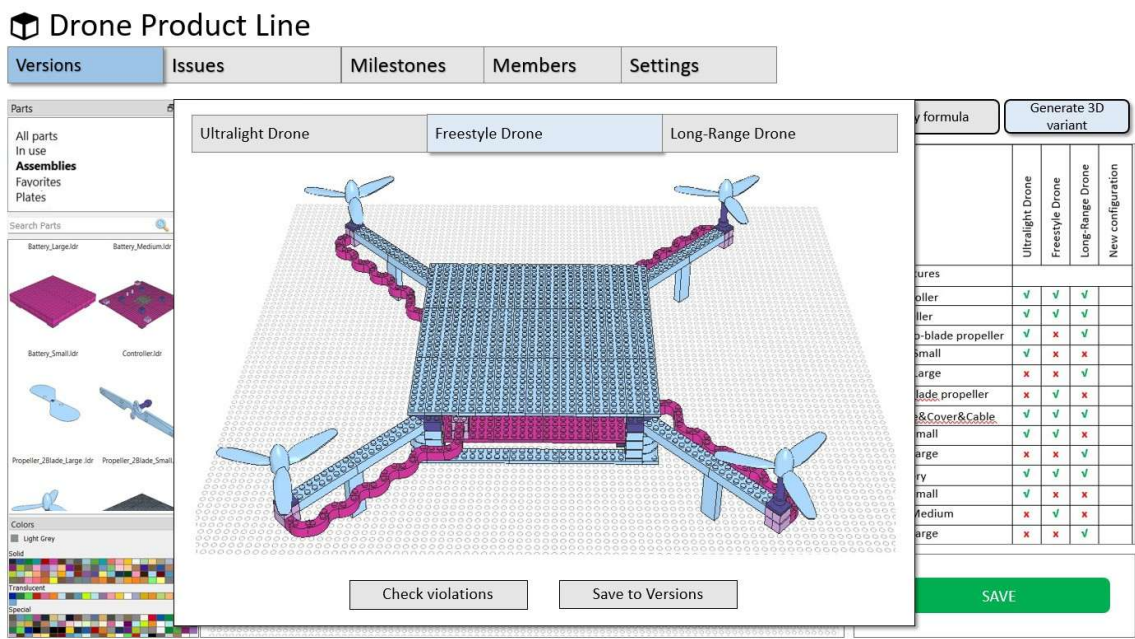


Figure 53: CADdrive's configuration space: generated 3D variant

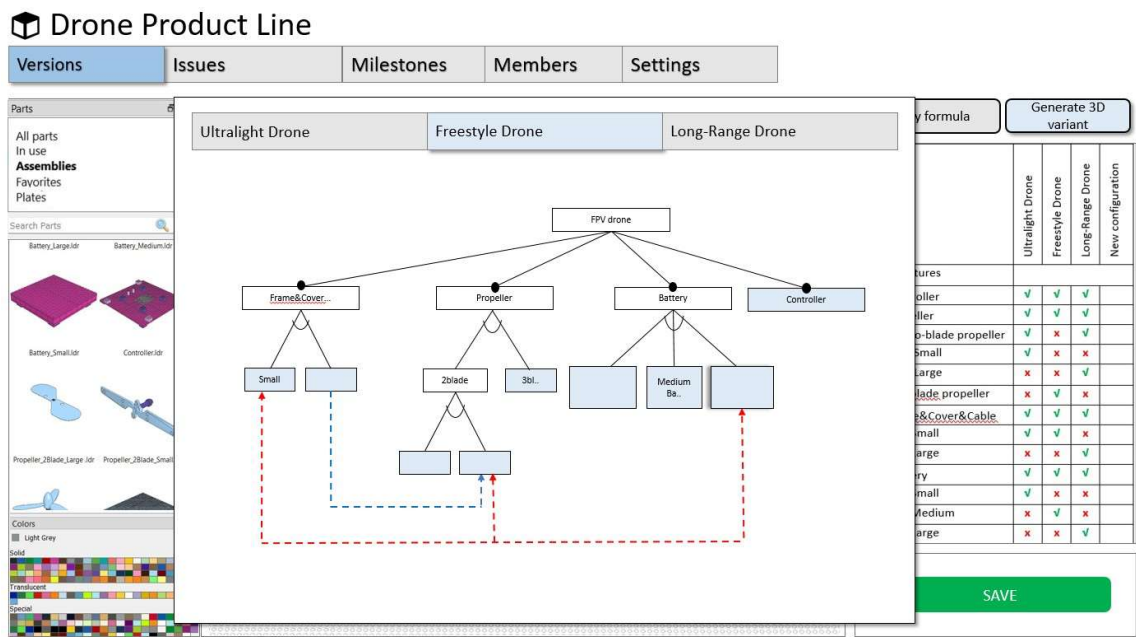


Figure 54: CADdrive's configuration space: generated feature model

5.6 MOCK-UP UI VERSIONS TAB

The UI Versions Tab is divided into three primary functional areas, as shown in Figure 55:

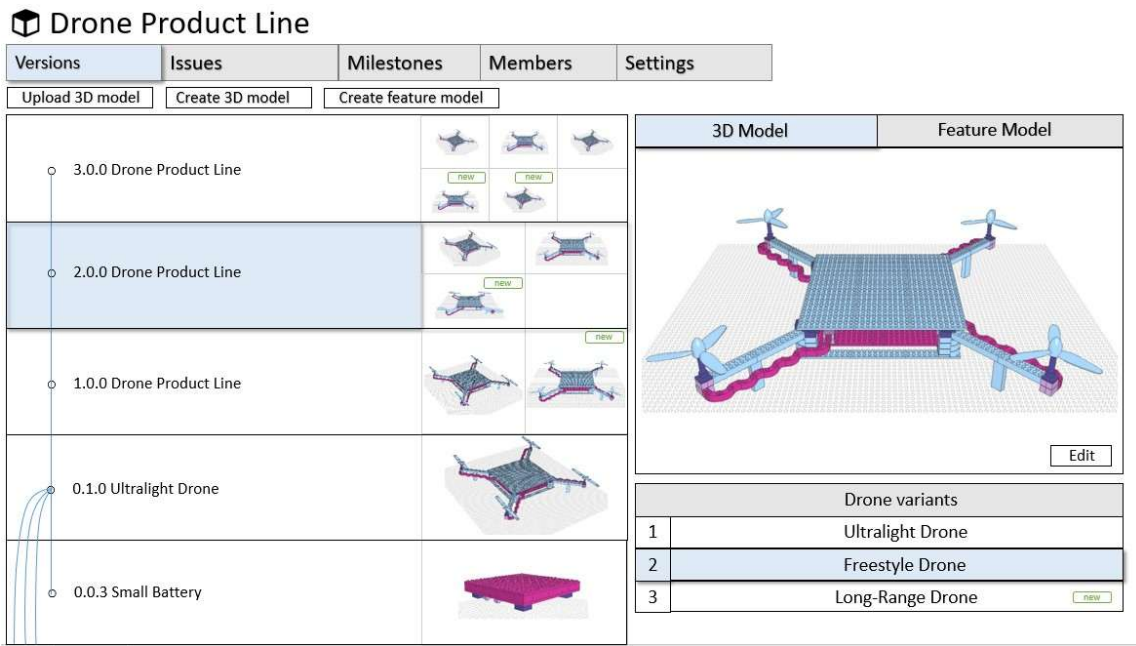


Figure 55: UI Versions Tab, 3D Model View

Top Navigation Bar

A horizontal menu provides access to core project management facets, including Versions, Issues, Milestones, Members, and Settings. The view presented focuses on the Versions aspect, central to the PLE process. It also enables users to upload 3D models, built in some other 3D modelling tools, create 3D models in CADdrive's configuration space or create variability models in its feature modelling space.

Left Panel

This panel provides a chronological record of the product line's development, listed from the most recent version (e.g., 3.0.0) to the oldest. Each version entry contains:

- A semantic version number.
- Icons representing the individual product variants contained within that release.

There is also a visual indicator (a green "new" label) attached to any variant that was introduced in that specific version. It offers immediate insight into the difference between consecutive releases.

Right Panel

This panel displays the architectural assets of the currently selected product line version. It comprises two synchronized views:

- 3D Model: A visualization of the product's physical geometry.
- Feature Model: A tree-structure representing features that are only relevant for a specific drone as it is shown in Figure 56.

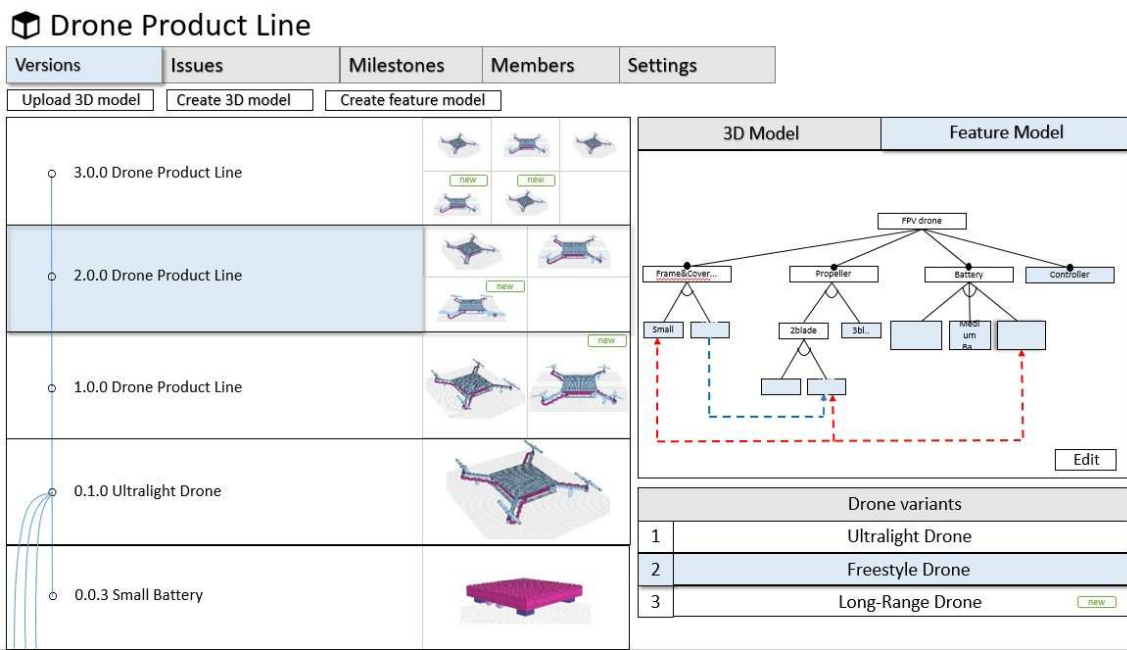


Figure 56: UI Versions Tab, Feature Model View

An "Edit" button serves as the entry point to the CADdrive's configuration space (described in 4.4.5), where users interact with a 150% model and the configuration table to define new configurations.

6 EVALUATION, DISCUSSION AND LIMITATIONS

6.1 CASE STUDY EVALUATION

The case study has undergone two iterations of refinement based on six anonymous reviews received during the SPLC 2025 submission process. The reviewers praised some aspects of the work and raised some questions that have significantly strengthened the manuscript, particularly regarding technical depth and methodological transparency.

6.1.1 POSITIVE FEEDBACK

These positive evaluations consistently highlighted following core strengths of the work:

Pedagogical value

Multiple reviewers explicitly endorsed LEGO's suitability for teaching PLE concepts, particularly praising its accessibility and reproducibility - key features for educational tools:

"LEGO is a good case study in general and especially well-suited for PLE education."
(Reviewer 2)

"Using Lego as an example provides an easy entry barrier. " (Reviewer 6)

"Interesting case study for teaching." (Reviewer 4)

Technical contribution

The focus on geometric constraints was recognized as addressing an important gap in PLE research, while the presentation quality supports effective knowledge transfer:

"Dealing with geometric/space constraints is a very challenging and important topic that has not been sufficiently addressed yet." (Reviewer 2)

"The integration of a visual design environment (LDraw) connects well with existing trends in low-code/no-code engineering." (Reviewer 6)

Originality

Reviewers confirmed the work's novelty in bridging digital/physical PLE applications, with the "ground truth" concept being particularly distinctive.

"The paper explores an interesting idea to build a reusable/reproducible case that could be used for PLE including hardware and software." (Reviewer 1)

"The idea of creating a reusable, fully accessible, and replicable ground truth is special." (Reviewer 2)

Methodology

Reviewers acknowledged the comprehensive literature review and clear research questions as strengths in establishing your scholarly foundation:

"The authors provide a research question as a guideline to orient their research objective" (Reviewer 6)

"Provides a comprehensive literature review of works relevant to the study's context" (Reviewer 5)

Presentation quality

The praise for figures and clarity suggests that the visual communication of complex concepts was especially effective:

"The work is well presented and easy to follow. Good figures." (Reviewer 2)

6.1.2 CRITICISM AND REFINEMENTS

The reviewers raised valuable critiques and in response to it, the work was significantly improved. These negative evaluations highlighted following weaknesses:

Soundness

"Why using this type of feature model? What are the reasons? There are many more representations and recent attempts have been made to define a unified variability-modeling language [4]; why not use that one instead to future proof the case?" (Reviewer 1)

and

"Why did the authors use an OVM when an FM can also be modeled? What advantages do OVMs bring in contrast to FMs in this context? Also, feature models could be more easily described, e.g., in UVL, which would make it more accessible to a larger community. To this end, the rationale behind choosing OVM instead of a feature model is unclear. In particular, UVL or other community-accepted formats could increase accessibility and reuse." (Reviewer 2)

and

"Consider replacing the outdated OVM with a more modern approach, such as decision models (IVML) or feature models (FeatureIDE or UVL)." (Reviewer 5)

The following improvements were undertaken:

Initially, an OVM was developed to model variability. However, after getting the peer feedback and evaluating alternatives (feature models, decision models and OVMs), this work employed a feature model. First, in contrast to OVMs, feature models uniquely represent both commonalities and variabilities in a single intuitive hierarchy. Second, their visual tree structure with explicit constraints makes variability relationships more accessible to students than table-based notations of decision models. Third, feature models could be more easily described, e.g., in UVL, which would make it more accessible to a larger community. Additionally, the ability to express feature models in standardized formats like UVL enhances accessibility for both educators and practitioners.

"Why have two distinct features frame and cover, if cover is fully depending on frame? Frame immediately defines cover, so is it necessary to split these up as conceptual features? Maybe yes, but I could not find a reasoning for it." (Reviewer 1)

The following improvements were undertaken: I

nitially modeled as separate features to reflect modular design principles, the frame and cover were later merged. Since the cover's geometry is entirely determined by the frame, treating it as a part of the frame improves and simplifies the variability model. This integration eliminates redundant constraints while maintaining configuration accuracy, and serves the educational purpose of demonstrating dependent component relationships in product line engineering.

"My biggest concern is the very small case study (only three product variants) and how the 150% model explicitly enumerates all product variants. The 150% model is actually an explicit enumeration." (Reviewer 3)

The following improvements were undertaken:

The product line comprising only three drones was deliberately designed as a compact teaching tool to introduce SPLE fundamentals without overwhelming students. However, in response to feedback, the 150% model was redesigned within the SPLE framework to properly implement variability points and configuration constraints rather than simple enumeration. While intentionally limited in scope, this approach effectively demonstrates core concepts and can be expanded with additional variants, maintaining the right balance between educational clarity and technical accuracy.

Transparency

"Shallow descriptions on how the drone example was built or from where it was inspired. The module and interface descriptions are not detailed enough to understand how they would influence the assembly of a product in a physical artifact." (Reviewer 5)

and

"How was the case study designed? What steps were involved in the analysis and construction of the product line? What were design decisions? How was the validity of the resulting artifact(s) evaluated? What are some metrics (number variants, lines of code, programming language, ...) of the product line, or is it only a feature model?" (Reviewer 1)

The following improvements were undertaken:

The descriptions of the drone's modular architecture have been significantly enhanced. The interfaces are now systematically categorized into mechanical and electrical/electromechanical types. This refinement improves the technical rigor of the case study while maintaining its pedagogical clarity. The revised version incorporates two key references that inspired the drone's design: Steiner et al.'s (2013) work on software product lines for drone systems, and modern MBSE practices (Samares Engineering, 2021) approaches to product line engineering. These foundational works informed both the conceptual framework and practical implementation of the case study. Furthermore, Chapter 4 has been expanded to include a comprehensive description of the component construction process, product line analysis and design decision rationale.

6.2 UI CONCEPT EVALUATION

The design of the user interface evolved through several iterations, guided by CADdrive's expert feedback. The initial conceptual approach proposed storing positional data (Cartesian coordinates and rotation angles) for each 3D part directly within the corresponding feature model element. In this paradigm, the system would generate a 3D model of a variant by assembling the configured components in a 3D space based on this stored geometric data. However, this approach was considered to be less than ideal because of its inherent complexity. The synchronization of the geometric data within the feature model resulted in significant overhead and potential for error, overcomplicating the modeling process.

Consequently, the design shifted towards a classical SPLE approach, leveraging a 150% model. This methodology requires users to manually construct a 150% 3D model. Variant generation then becomes a subtractive process, so that the system removes or hides components that are not relevant for a specific feature configuration. To implement this, an initial solution proposed the creation of three libraries: one for 3D parts, a second for feature model elements, and a third "mapping library" to define the relationships between them. While feasible, this multi-library structure introduced unnecessary complexity in data management.

A more elegant solution was subsequently identified: storing the feature model information directly within the existing LDraw file structure using metadata and naming conventions. This integration significantly reduces architectural complexity by unifying the data model, thereby streamlining the entire system and simplifying the user interface. The final graphical user interface concept was developed based on this integrated architecture. Furthermore, the visual design and interaction patterns were refined across multiple iterations based on expert evaluations focusing on both functional clarity and aesthetic usability.

6.3 ANSWERS TO RESEARCH QUESTIONS

6.3.1 RESEARCH QUESTION 1

How could SPLE techniques be adapted for digital LEGO?

SPLE techniques can be easily adapted for digital LEGO by leveraging its inherent properties as a modular, constraint-driven modeling system. The main idea is, to use digital LEGO bricks as the reusable assets of a product line, where their standardized stud-and-tube interfaces naturally define compatibility rules. The hierarchical assembly structure of the LDraw file format directly supports the organization of a core asset base, allowing complex products to be built from smaller, configurable modules. Variability is managed by mapping different LEGO components to features in a variability model, while geometric and spatial constraints—such as collision detection between a large propeller and a small frame—are explicitly modeled to enforce valid configurations. Finally, the

derivation process is automated by implementing an algorithm that filters the comprehensive 150% LEGO model based on a selected feature configuration, hiding irrelevant components to generate a specific product variant.

6.3.1.1 RESEARCH QUESTION 1.1

How could a variability model look like?

Variability is represented through a feature model, chosen for its intuitive, hierarchical tree representation over decision models and orthogonal variability models. This graphical model organizes the product line's options into a hierarchical tree, defining mandatory features, optional choices, and sets of mutually exclusive alternatives, all linked by "requires" and "excludes" constraints. A key contribution of this work is the translation of this graphical model into a structured textual formula. This formula encodes the entire hierarchy and its constraints using a specific naming convention. It enables the direct integration of the variability logic with the 150% model. By embedding this formula into the LDraw file's structure, the abstract feature definitions are linked to their physical 3D components.

6.3.1.2 RESEARCH QUESTION 1.2

How could a LEGO-based 150% model look like?

A LEGO-based 150% model incorporates all the possible atomic modules in one 3D space. Moreover, the model integrates the information from a variability model directly into its structure. This is done by using a structured naming convention for LDraw submodels, where their names encode feature information, constraints, and hierarchical relationships. The model contains both concrete features, which are represented by 3D elements, and abstract features, which exist only as organizational nodes within the hierarchy to define the configuration rules. This approach merges the traditional feature diagram with the geometrical representation, creating a unified source that contains all necessary information for defining and deriving any valid product variant.

6.3.1.3 RESEARCH QUESTION 1.3

How to derive a 3D model of a configuration from the 150% model automatically?

A 3D model is derived automatically from the 150% model through a configuration-driven filtering algorithm. The process begins with a user-selected configuration as an input. The algorithm then goes through the hierarchical tree structure of the LDraw file, which has been preconfigured with integrated feature information using a specific naming convention. During this process, each node is processed based on its metadata. Mandatory nodes are retained and their children are processed recursively. For optional nodes, the algorithm checks if their associated feature is present in the selected configuration. If yes, the node and its subtree are processed and kept depending on the conditions described in the formula, otherwise, they are removed. The Boolean operators OR, XOR, and AND are applied to ensure the correct number of child features are selected. The output of this process is a filtered 3D model where all components not belonging to the chosen variant have been removed.

6.3.2 RESEARCH QUESTION 2

How to integrate the framework into CadDrive?

The SPLE framework is integrated into CADdrive by adding three dedicated UI components. A Feature Model Tab enables the creation of variability models. A Configuration Space allows users to create 150% models, map features to 3D components of 150% models, select configurations, and automatically generate variant models. Finally, a Versions Tab manages the product line's evolution, displaying synchronized feature models and 3D assemblies for each variant of each version. This integration is supported by storing feature model data directly within the LDraw files.

6.4 LIMITATIONS

Despite the fact that the research was conducted with methodological rigor, certain limitations should still be considered.

6.4.1 REDUCED UTILISATION OF THE SPLE FRAMEWORK

In the thesis, the SPLE framework is used to configure a product only out of 3D models. Due to the main focus on 3D modelling and specificity of CADdrive, other assets which are critical in drones' development, e.g. requirements, lines of codes, technical drawings, simulation models and test results, were not configured. However, to employ the full potential of SPLE framework, it is essential to consider these assets.

6.4.2 LACK OF FORMILIZED GUIDELINES

No process to derive similar case studies in other domains has been presented for adapting the SPLE methodology. The framework's current state serves best as a template for similar small-scale educational cases rather than a ready-made solution for a random physical product line.

6.4.3 LACK OF STATISTICAL OR COMPARATIVE DATA

Only one CADdrive expert participated in functionality testing to define which version of the prototype performs better. No larger multifunctional teams tested the framework. Full A/B test with comparing different buttons, headlines and images was not employed. This thesis provide only qualitative data and does not have any team-wide surveys or real-world analytics.

6.4.4 ONLY CONCEPTUAL EVALUATION

The focus of the thesis is on conceptual integration of the SPLE framework into CADdrive. Although SPLE and CADdrive experts were questioned in the process, no student team or industry engineers were participating. Utilization in the real-world environment may discover constraints and usability issues.

6.4.5 ABSTRACTION FROM SMALL-SCALE AND REPETITIVE COMPONENTS

The thesis demonstrates the deliberate abstraction from small-scale, standardized, and highly repetitive components, such as screws, nuts, and connectors. The proposed framework operates at the level of architecturally significant modules that define a product

variant's core identity and functionality. Representing every individual screw as a feature in the variability model or as a component in the 150% LEGO model is neither practical nor pedagogically useful, as it would add too much complexity.

The small-scale highly repetitive elements are better managed within a detailed BOM linked to each module's internal design. Consequently, the framework is ideally suited for teaching high-level variant configuration, architectural design, and interface management, but it does not replicate the full granularity of a production-ready engineering BOM, which remains a focus for future industrial application.

7 CONCLUSION AND FUTURE RESEARCH

7.1 SUMMARY

This thesis successfully conceptualized the integration of SPLE principles into the open-source educational platform CADdrive, using digital LEGO as a low-fidelity modeling tool. A key finding of this research is the development of an approach, a formula, which enables the integration of the abstract variability model into the concrete 150% LEGO model. This formula, integrated directly within the LDraw file's hierarchical structure, encodes feature types, constraints, and hierarchical relationships, enabling the automated derivation of product variants from a single 150% model.

The primary pedagogical value of this work lies in its ability to make complex SPLE concepts tangible and accessible. By leveraging the familiarity of LEGO and a visual, hands-on approach, the framework lowers the entry barrier for students, allowing them to directly experiment with variability modeling, modular architecture, and configuration dependencies without the overhead of complex industrial tools. The research demonstrated its viability through a structured evaluation and validation process. The core conceptual framework was refined through expert peer review, which confirmed its pedagogical value, originality in addressing geometric constraints, and technical soundness. Furthermore, the proposed user interface prototype was iteratively developed and validated with a CADdrive expert.

Despite certain limitations—such as the conceptual nature of the prototype and its focus on 3D models—the framework provides a practical, accessible foundation for teaching modular design and variant management. The formula-based integration of the feature and 150% model effectively bridges the gap between theoretical SPLE concepts and their application in physical product development, offering a reproducible "ground truth" for educational use.

7.2 FUTURE RESEARCH

Several potential directions for future research are identified:

Empirical Validation with End-Users: The next logical step after the CADdrive expert feedback would comprise creating a clickable UI prototype and conducting studies with

student teams to evaluate the usability and learning effectiveness of the approach and the prototype.

Expansion of Asset Types: Future work should investigate how the approach can be extended beyond 3D models to include other important product assets, such as technical documentation, BOMs and test cases etc., creating a more comprehensive learning environment. That would enhance the industrial relevance of the framework and provide a better learning path for students.

Development of Formalized Guidelines: Creating a structured methodology and ready-to-use teaching materials would help educators adopt this framework and increase its impact.

8 REFERENCES

- Abbas, M., Jongeling, R., Lindskog, C., Enoiu, E., Saadatmand, M., & Sundmark, D. (2020). Product line adoption in industry: An experience report from the railway domain. *International Systems and Software Product Line Conference, A*, 1–11.
- Baldwin, C., & Clark, K. (2003). Managing in an age of modularity. In R. Garud, A. Kumaraswamy, & R. Langlois (Eds.), *Managing in the modular age: Architectures, networks, and organizations* (Issue 149, pp. 84–93). Wiley.
- Campbell, G., Faulk, S., & Weiss, D. (1990). Introduction To Synthesis. *Tech. Rep. INTRO_SYNTHESIS_PROCESS-90019-N*. Software Productivity Consortium, Herndon, VA, USA.
- Clements, P., & Northrop, L. (2002). *Software Product Lines: Practices and Patterns*. Addison-Wesley.
- DeepSeek (2025). <https://chat.deepseek.com>
- Doma, O. O., & Şener, S. M. (2021). *Using Modular Construction Brick-Based CAD in Online Design Education*.
- Dube, S. (2021). *Product Line Engineering and Model Based Systems Engineering – Overview*. Samares Engineering. <https://www.samares-engineering.com/en/2021/07/19/product-line-engineering-and-model-based-systems-engineering-overview>
- Edson, O., Itana, G., & Elisa, H. (2005). *A variability management process for software product lines*.
- Feichtinger, K., Sundermann, C., Thüm, T., & Rabiser, R. (2022). It’s your loss: Classifying information loss during variability model roundtrip transformations. *Proceedings of the 26th ACM International Systems and Software Product Line Conference - Volume A*, 67–78. <https://doi.org/10.1145/3546932.3546990>
- Flores, R., Krueger, C., & Clements, P. (2012). Mega-scale product line engineering at General Motors. *Proceedings of the 16th International Software Product Line Conference - Volume 1*, 259–268. <https://doi.org/10.1145/2362536.2362571>
- Góngora, H. G. C., Ferrogallini, M., & Moreau, C. (2015). How to Boost Product Line Engineering with MBSE - A Case Study of a Rolling Stock Product Line. In F. Boulanger, D. Krob, G. Morel, & J.-C. Roussel (Eds.), *Complex Systems Design &*

Management (pp. 239–256). Springer International Publishing.
https://doi.org/10.1007/978-3-319-11617-4_17

Gregg, S. P., Hartley, D., McAfee, M., Pitz, R., Teaff, J., & Clements, P. (2020). Patterns for Success in the Adoption and Execution of Feature-based Product Line Engineering: A Report from Practitioners. *INCOSE International Symposium*, 30(1), 465–482.
<https://doi.org/10.1002/j.2334-5837.2020.00734.x>

Gregg, S., Scharadin, R., & Clements, P. (2015). The more you do, the more you save: The superlinear cost avoidance effect of systems product line engineering. *International Systems and Software Product Line Conference, A*, 303–310.

Grogan, P., & De Weck, O. (2019). LEGO Product Design and Manufacturing Simulations for Engineering Design and Systems Engineering Education. *2019 ASEE Annual Conference & Exposition Proceedings*, 33056. <https://doi.org/10.18260/1-2--33056>

Hackenberg, G., & Zehetner, C. (2025). *CADdrive Products*. CADdrive.com.
<https://caddrive.com/products>.

Kang, K. C., Cohen, S. G., Hess, J. A., Novak, W. E., & Peterson, A. S. (1990). *Feature-Oriented Domain Analysis (FODA) Feasibility Study*. US Dept of the Air Force.
<https://doi.org/10.21236/ada235785>

Kästner, C., & Apel, S. (2013). Feature-Oriented Software Development. In *Generative and Transformational Techniques in Software Engineering IV* (Vol. 7680, pp. 346–382). Springer, Berlin, Heidelberg.

Koch, T., Holtmann, J., Schubert, D., & Lindemann, T. (2016). Towards Feature-based Product Line Engineering of Technical Systems. *Procedia Technology*, 26, 447–454.
<https://doi.org/10.1016/j.protcy.2016.08.057>

Krueger, C., & Clements, P. (2013). Systems and software product line engineering. *Encyclopedia of Software Engineering*, 2, 1–14.

Kuiter, E., Thüm, T., & Kehr, T. (2025). Teach Variability! A Modern University Course on Software Product Lines. *Proceedings of the 19th International Working Conference on Variability Modelling of Software-Intensive Systems*, 1–10.
<https://doi.org/10.1145/3715340.3715441>

- Kurkovsky, S., Ludi, S., & Clark, L. (2019). Active Learning with LEGO for Software Requirements. *Proceedings of the 50th ACM Technical Symposium on Computer Science Education*, 218–224. <https://doi.org/10.1145/3287324.3287444>
- Lameh, J., Dubray, A., & Jankovic, M. (2023). TOWARDS A CONFIGURATION MANAGEMENT INTEGRATION TO FEATURE MODELS IN MODEL-BASED PRODUCT LINE ENGINEERING. *Proceedings of the Design Society*, 3, 3581–3590. <https://doi.org/10.1017/pds.2023.359>
- Li, H., Yang, M., & Evans, S. (2019). Classifying different types of modularuty for technical system. *International Journal of Technology Management*, 81(1/2), 1–23.
- Linden, F. van der, Schmid, K., & Rommes, E. (2007). *Software Product Lines in Action: The Best Industrial Practice in Product Line Engineering*. Springer.
- Lopez-Fernandez, D., Gordillo, A., Ortega, F., Yague, A., & Tovar, E. (2021). LEGO® Serious Play in Software Engineering Education. *IEEE Access*, 9, 103120–103131. <https://doi.org/10.1109/ACCESS.2021.3095552>
- Mebane, H., & Ohta, J. (2007). Dynamic Complexity and the Owen Firmware Product Line Program. *International Systems and Software Product Line Conference*, 212–222.
- Parslov, J. F., & Mortensen, N. H. (2015). Interface definitions in literature: A reality check. *Concurrent Engineering*, 23(3), 183–198. <https://doi.org/10.1177/1063293x15580136>
- Piran, F. A. S., Lacerda, D. P., Antunes, J. A. V., Viero, C. F., & Dresch, A. (2016). Modularization strategy: Analysis of published articles on production and operations management (1999 to 2013). *The International Journal of Advanced Manufacturing Technology*, 86(1–4), 507–519. <https://doi.org/10.1007/s00170-015-8221-9>
- Pohl, K., Böckle, G., & Linden, F. (2005). *Software Product Line Engineering: Foundations, Principles, and Techniques*. Springer.
- Prince, M. J., & Felder, R. M. (2006). Inductive Teaching and Learning Methods: Definitions, Comparisons, and Research Bases. *Journal of Engineering Education*, 95(2), 123–138. <https://doi.org/10.1002/j.2168-9830.2006.tb00884.x>
- Romano, D., Feichtinger, K., Beuche, D., Ryssel, U., & Rabiser, R. (2022). Bridging the gap between academia and industry: Transforming the universal variability language to

pure::variants and back. *Proceedings of the 26th ACM International Systems and Software Product Line Conference - Volume B*, 123–131. <https://doi.org/10.1145/3503229.3547056>

Schmid, K., & Verlage, M. (2002). The economic impact of product line adoption and evolution. *IEEE Software*, 19(4), 50–57. <https://doi.org/10.1109/ms.2002.1020287>

International Organization for Standardization (2015). *ISO 26550:2015 Software and systems engineering—Reference model for product line engineering and management*.

Steiner, E., Masiero, P., & Bonifácio, R. (2013). Managing SPL Variabilities in UAV Simulink Models with Pure::variants and Hephaestus. *CLEI Electronic Journal*, 16(1). <https://doi.org/10.19153/cleij.16.1.6>

Trigaux, J.-C., & Heymans, P. (2003). *Software Product Lines: State of the art*.

Ulrich, K. (1995). The role of product architecture in the manufacturing firm. *Research Policy*, 24(3), 419–440.

Wang, C.-S. (2009). Web-based modular interface geometries with constraints in assembly models. *Computers & Industrial Engineering*, 56(4), 1675–1686.

Warniez, A., Penas, O., Plateaux, R., & Soriano, T. (2014). SysML geometrical profile for integration of mechatronic systems. *2014 IEEE/ASME International Conference on Advanced Intelligent Mechatronics*, 709–714. <https://doi.org/10.1109/aim.2014.6878162>

Young, D. B., & Clements, D. P. (2017). Model Based Engineering and Product Line Engineering: Combining Two Powerful Approaches at Raytheon. *27th Annual INCOSE International Symposium (IS 2017)*, 518–532.